

Biff Notification System: A Z Specification

Formal Model of Push Notification State Transitions

March 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	3
4	Global Constants	3
5	State	4
6	Initialization	6
7	Session Capture	7
8	NATS Connection Operations	8
8.1	Ensure Connected	8
8.2	Close Connection	9
9	Tool Context	10
10	Notification Core	12
10.1	Belt Path (Tool Context Available)	12
10.2	Suspenders Path (No Tool Context), idealised	16
10.3	Total Notification	17
11	Send Flow Operations	18
11.1	Wall Post (Tool Handler)	18
11.2	Talk Start (Tool Handler)	19
11.3	External Message Deliver (CLI)	19
12	Receive Flow Operations	20
12.1	KV Wall Receive	20
12.2	NATS Talk Callback	21
12.3	Sync Unread File	22
12.4	Poll Tick (Background Poller)	23
13	Suspenders Failure and the No-Recovery Sink	24
13.1	Recovery: a mid-session reconnect	28
13.2	Capture/Reconnect Flush	29
13.3	The counterexample	33
14	The Race: Three Critical Sections, One Shared Flag	35
14.1	Status Bar Refresh	37
15	Display Queue Operations	38
15.1	Advance If Due	38

16 Tool-Description Marker Channels	40
16.1 Read Channel (the self-clearing reference)	41
16.2 Talk Channel (invite, message, drain)	42
17 Marker Consistency: the property and the defect	47
18 The Fix: reconciling transitions	48
18.1 Draining messages and ending the conversation	49
18.2 Withdrawing a pending invite (ntWithdraw frame)	49
18.3 A withdrawal that names the wrong session (WithdrawForeign)	51
18.4 The withdrawal gap, as a dead operation (WithdrawStale)	52
18.5 Expiring a pending invite (time-to-live)	53
18.6 Accepting an invite	55
19 Marker Consistency: the proof	56
20 System Properties	58
20.1 Notification Delivery Guarantee	58
20.2 Session Invariant—retracted	58
20.3 Suspenders Always Available—retracted	58
20.4 Wall and Talk Symmetry	59
20.5 Startup Sequence	60

1 Introduction

Biff’s push notification system delivers real-time updates (wall broadcasts and talk messages) to Claude Code’s status bar through two independent channels: the MCP protocol (fast, tool-list-changed notification) and a filesystem status file (slow, polled by the status bar command).

The notification pipeline has a “belt and suspenders” design: tool handlers fire notifications via the MCP request context (belt path), while a background poller fires them via a stored `ServerSession` reference (suspenders path). NATS subscription callbacks and KV watchers only wake the poller from napping; the poller detects the changes on its next active tick and fires the notification from the MCP server’s own coroutine context.

The session reference is captured eagerly during the server’s lifespan startup, before the NATS connection is established and before the background poller begins—this is what closes the biff-8g0 defect class, in which the poller could wake before any session had ever been captured. NATS callbacks and KV watchers do not fire notifications directly—they are unreliable from background coroutine contexts—but they wake the poller from napping, so the next active tick (2 seconds) detects the changes via its own reads and attempts the notification.

Whether that attempt succeeds is a separate question from whether a session was ever captured, and Sections 5 and 13 correct an over-claim in earlier drafts on exactly this point. A send on the stored session reference can fail independently of NATS connectivity—the client’s own MCP transport can die while the NATS connection underneath stays healthy—and root cause #1 (“dropped tools/list_changed mid-session, no recovery,” Section 13) is what happens when that failure is not recorded: the session reference is cleared, the description-changed flag is left set, and nothing tells the next client reconnect that a notification is owed. The governing property is not “the poller always has a valid session”—that is false, once a send can fail—but the accounting safety property *notifyLost = zfalse*: every suspenders failure is recorded, so recovery is guaranteed rather than accidental.

This specification models the four state spaces—NATS connection, MCP server session, filesystem, and status bar—and the operations that transition them during the wall and talk notification flows.

A second delivery mechanism sits alongside the status bar: the server surfaces incoming activity by mutating a *tool description* and firing `tools/list_changed`. The `read_messages` description gains (N unread); the `talk` description gains a `[TALK] ...` banner with an accept hint. Sections 16 to 19 model these two channels and establish the governing invariant: for each channel the marker is present exactly when its activity exists, and a talk marker always carries a session-naming accept hint. The read channel is self-clearing and serves as the reference; the talk channel, as originally built (biff-9la), was not. The stale-marker defect is reproduced here as a reachable ProB state and then proved unreachable once the reconciling transitions of Section 18 are in place.

A third defect sits underneath both marker channels rather than alongside them: the marker can be server-side correct—mutated, consistent with *MarkerConsistent*—while the notification that would tell Claude Code to re-read it is silently lost. Section 13 models this as root cause #1, reproduces it as a reachable ProB state, and proves the corresponding safety property once the suspenders failure is properly recorded. A second round of review found the same failure mode at the other two sites that write *pendingNotify*—the capture/reconnect flush (Section 13.2) and the belt path itself (Section 10.1)—and, sharper still, found that treating each site’s read-then-await-then-write as atomic was itself the coarseness: Section 14 splits all three into a read/decide step and a write step, reproduces the resulting lost-update race as a reachable state, and proves it closed by a single mutex serialising the three critical sections.

2 Basic Types

[*USER*, *MSG*, *SOURCEKEY*, *SESSIONKEY*]

USER identifies a biff user. *MSG* is an opaque message body. *SOURCEKEY* uniquely identifies a display queue item (e.g. `wall:2026-03-01T12:00:00` or `talk:eric`). *SESSIONKEY* is a composite `user:tty` key naming one specific session; it is the same notion as the *SESSIONKEY* of *talk.tex*, and it is what an accept must target (`talk @user:tty`). A pending talk invite that retains its inviter’s *SESSIONKEY* can render an accept hint that names a session; one that has dropped it can only render a bare `@user`, which is not a runnable accept command.

3 Free Types

$ZBOOL ::= ztrue \mid zfalse$

$NatsState ::= natsDisconnected \mid natsConnected \mid natsClosed$

The NATS client connection lifecycle: initially disconnected, lazily connected on first use, and permanently closed on shutdown.

$SessionState ::= sessNone \mid sessCaptured$

The MCP `ServerSession` reference: either not yet captured (`sessNone`) or captured during server lifespan startup (`sessCaptured`). Once captured, the session reference is never invalidated.

$ItemKind ::= itemWall \mid itemTalk$

Display queue items are either persistent wall announcements or ephemeral talk messages.

$NotifyResult ::= notifySent \mid notifyDropped$

The outcome of a `notify_tool_list_changed` attempt. *Erratum*: earlier drafts claimed `notifyDropped` was only the initial default value and that every notification-triggering operation produces `notifySent`. That is false once a send can genuinely fail (Section 13): `notifyDropped` is a real, reachable outcome at all three sites—suspenders, belt, and the capture/reconnect flush. What holds instead is `notifyLost = zfalse`: every site that produces `notifyDropped` also re-arms `pendingNotify`, so a dropped send is recorded rather than silently absorbed.

$CriticalSite ::= siteSuspenders \mid siteBelt \mid siteCapture$

The three call sites that write `pendingNotify` as the consequence of an awaited send: the background poller (Section 13), the belt path (Section 10), and the capture/reconnect flush (Sections 7 and 13). Section 14 splits each into a read/decide step and a write step and uses `CriticalSite` to track which sites currently have a write outstanding between the two.

4 Global Constants

The display queue and the unread counter are bounded. In the running system the display rotation queue holds at most twenty items and the unread count is capped at one hundred. Pinning those concrete numbers into the model, however, makes exhaustive checking hopeless: a sequence of up to twenty source keys over even two carriers, crossed with an unread count ranging over 0 .. 100, yields hundreds of thousands of reachable states before any interesting property is reached.

Following the convention of the sister specification *talk.tex*, the bounds are therefore not hard-wired. Each is the cardinality of an abstract carrier—*QSL*OT for queue slots, *USL*OT for unread slots—so the bound is a *scoping parameter* ProB shrinks for exhaustive checking rather than a fixed value. A third carrier, *ASL*OT, plays the same role for the pending-invite time-to-live: *maxInviteAge* is its cardinality, and a pending invite whose age reaches that bound may be aged out. A fourth carrier, *PSL*OT, bounds the pending-invite set itself: *maxPending* is its cardinality, the number of pending invites the recipient will retain at once. This bound is not decoration. A pending invite is keyed by the wire-controlled inviter identity; the time-to-live is eventual, not a cap, so without a size bound each distinct forged inviter would add a permanent entry, an unbounded-growth denial of service (the biff-vr4 analogue). The bound caps the set exactly as *maxQueueSize* caps the message queue, and *TalkInviteArriveOverflow* evicts to hold it, mirroring *ReceiveOverflow* of *talk.tex*. All four carriers are required non-empty, so each bound is at least one; at *#ASL*OT = 1 the clock has a single tick, which is the smallest model in which a fresh invite (age 0) and an expired one (age 1) are distinct.

$[QSL\text{OT}, USL\text{OT}, ASL\text{OT}, PSL\text{OT}]$

$maxQueueSize : \mathbb{N}$ $maxUnreadCount : \mathbb{N}$ $maxInviteAge : \mathbb{N}$ $maxPending : \mathbb{N}$	$QSL\text{OT} \neq \emptyset$ $USL\text{OT} \neq \emptyset$ $ASL\text{OT} \neq \emptyset$ $PSL\text{OT} \neq \emptyset$ $maxQueueSize = \#QSL\text{OT}$ $maxUnreadCount = \#USL\text{OT}$ $maxInviteAge = \#ASL\text{OT}$ $maxPending = \#PSL\text{OT}$
--	--

Default descriptions and the idle marker are opaque constants:

$wallDescDefault : MSG$ $talkDescDefault : MSG$ $readDescDefault : MSG$ $idleMarker : MSG$	
---	--

$readDescDefault$ is the base `read_messages` tool description, carrying no unread marker. Its role in the read channel mirrors that of $talkDescDefault$ in the talk channel and $wallDescDefault$ in the wall channel.

The intended verification is exhaustive model checking at set size 2, which supplies two distinct carrier elements throughout. Modelling the tool-context brackets (Section 9) roughly doubles the reachable state count and, by making the belt path live, opens the receive-flow subspace behind $TalkStart$; at the full queue and unread bounds the check no longer terminates in a practical time. The bounded carriers exist precisely for this: shrinking $QSL\text{OT}$, $USL\text{OT}$ and $ASL\text{OT}$ to one keeps the exploration complete while retaining every distinction the properties depend on.

```
probccli docs/notification.tex -model_check -noltl -coverage \
  -p DEFAULT_SETSIZE 2 \
  -card SOURCEKEY 1 -card SESSIONKEY 1 \
  -card QSL\text{OT} 1 -card USL\text{OT} 1 -card ASL\text{OT} 1 -card PSL\text{OT} 1 \
  -p MAX_INITIALISATIONS 16 -p MAX_OPERATIONS 8000
```

Each carrier may be shrunk independently with `-card X n` without disturbing the message and key carriers. $MAX_INITIALISATIONS$ must exceed the number of identities $Init$ leaves unconstrained, or ProB silently drops initial states and reports an incomplete exploration; at these bounds $Init$ is fully determined, so exploration reports $INITIALISATION:8$ against $SETUP_CONSTANTS:8$ with the bound never binding, and terminates with “all open states visited”—a complete check, not a truncated one.

5 State

The system state is flattened into a single schema for ProB compatibility. It covers four subsystems: NATS connection state, MCP session capture, the display queue, the filesystem status file, the status bar, and the last notification outcome.

NotificationState

natsConn : NatsState
kvProvisioned : ZBOOL
talkSubscribed : ZBOOL
session : SessionState
toolContext : ZBOOL
wallDesc : MSG
talkDesc : MSG
readDesc : MSG
wallDescChanged : ZBOOL
talkDescChanged : ZBOOL
unreadCount : $\mathbb{N}$
talkPending : $USER \rightarrow SESSIONKEY$
talkBare : $\mathbb{P} \text{ USER}$
talkQueued : $\mathbb{N}$
talkConnected : ZBOOL
pendingAge : $\mathbb{N}$
queueItems : seq SOURCEKEY
queueKinds : SOURCEKEY $\rightarrow$ ItemKind
queueTexts : SOURCEKEY $\rightarrow$ MSG
currentIndex : $\mathbb{N}$
fileExists : ZBOOL
fileUnreadCount : $\mathbb{N}$
fileItems : seq SOURCEKEY
barLine2 : MSG
barStale : ZBOOL
lastNotifyResult : NotifyResult
staleWithdrawHit : ZBOOL
pendingNotify : ZBOOL
notifyLost : ZBOOL
inFlight : $\mathbb{P} \text{ CriticalSite}$
beltPnSnapshot : ZBOOL
capPnSnapshot : ZBOOL
raceLost : ZBOOL

dom *queueKinds* = ran *queueItems*
dom *queueTexts* = ran *queueItems*
#*queueItems* $\leq$ maxQueueSize
(*queueItems* $\neq \langle \rangle$ $\Rightarrow$ *currentIndex* $<$ #*queueItems*)
(*queueItems* = $\langle \rangle$ $\Rightarrow$ *currentIndex* = 0)
fileUnreadCount $\leq$ maxUnreadCount
unreadCount $\leq$ maxUnreadCount
talkQueued $\leq$ maxQueueSize
#*talkPending* + #*talkBare* $\leq$ maxPending
pendingAge $\leq$ maxInviteAge
((dom *talkPending* = $\emptyset$ $\wedge$ *talkBare* = $\emptyset$) $\Rightarrow$ *pendingAge* = 0)
dom *talkPending* $\cap$ *talkBare* = $\emptyset$
((*talkDesc* $\neq$ *talkDescDefault*) $\Leftrightarrow$
 (dom *talkPending* $\neq \emptyset \vee$ *talkBare* $\neq \emptyset \vee$
 talkQueued $> 0 \vee$ *talkConnected* = ztrue))
((*readDesc* $\neq$ *readDescDefault*) $\Leftrightarrow$ *unreadCount* > 0)
(*talkDesc* $\neq$ *talkDescDefault* $\Rightarrow$ *talkBare* = $\emptyset$)
(*talkSubscribed* = ztrue $\Rightarrow$ *natsConn* = *natsConnected*)
(*kvProvisioned* = ztrue $\Rightarrow$ *natsConn* = *natsConnected*)
staleWithdrawHit = zfalse
notifyLost = zfalse
#*inFlight* ≤ 1
raceLost = zfalse

Key invariants: queue metadata maps have the same domain as the queue sequence range; subscription and KV require a live connection; the current index is always within bounds; the pending-invite set is bounded, $\#talkPending + \#talkBare \leq maxPending$, so no stream of forged inviters can grow it without limit; and $notifyLost = zfalse$, established in Section 13, which says that a send failure at any of the three *pendingNotify*-writing sites is never silently absorbed. The last two, $\#inFlight \leq 1$ and $raceLost = zfalse$, are Section 14's: the first is the Z-level model of a mutex serialising the three sites' critical sections, and the second says that under it no site's write can ever land on a stale read of another site's concurrent write.

Erratum (root cause #1, dropped tools/list_changed mid-session). Earlier drafts of this section carried a further invariant, $natsConn = natsConnected \Rightarrow session = sessCaptured$, glossed as “a connected NATS client implies a captured session,” with the accompanying claim in Section 7 that “once captured, the session reference is never invalidated.” That claim is false of the running system. *notify_tool_list_changed()*'s suspenders branch clears the stored session reference to `None` when a send raises—a live NATS connection survives the send failure untouched, but the cached session reference does not—so $natsConn = natsConnected$ and $session = sessNone$ can co-occur in the real system. The invariant as stated is refuted by exactly the transition that models this failure (*PollTickNotifyFail*, Section 13); keeping it would make that transition infeasible and hide the defect rather than expose it. It is removed here. The property it was protecting—that *EnsureConnected* can only run after a session has been captured at least once, the biff-8g0 guard—did not actually depend on session capture being permanent; it depended only on capture happening *before the first connect*. That is now stated as an explicit precondition on *EnsureConnected* itself (Section 8.1), where it belongs, rather than as a state invariant that also asserts something stronger and false.

The tool-description channels. Three of the components above model tool descriptions that Claude Code shows to the agent, and two more model the underlying activity that a description marker is supposed to track. These are the subject of the correctness argument in Section 16.

readDesc is the `read_messages` tool description; it gains a (N `unread`) marker when mail is pending. Its source of truth is *unreadCount*, the server-side count of unread messages, which *decreases* as mail is read—so a description tied to it clears itself. This is the reference channel: it is known to behave correctly, and it is modelled here so that the talk channel can be judged against a worked example rather than against prose.

talkDesc is the `talk` tool description; it gains a [TALK] ... marker, including an accept hint, when a talk invite, queued message, or live connection exists. Its activity has three sources: *talkPending*, a partial function mapping each user who has invited us to that invite's originating *SESSIONKEY*; *talkQueued*, the count of undrained talk messages; and *talkConnected*, whether a conversation is live. A pending invite in *talkPending* retains the inviter's session key, so its accept hint can name a session.

talkBare is the pathological representation: a set of users whose pending invite was recorded *without* a session key—the tty was dropped. A user in *talkBare* is genuine activity (the marker appears), but its accept hint can only read `talk @user` with no `:tty`, which is not a runnable accept command. The structural invariant $\text{dom } talkPending \cap talkBare = \emptyset$ says a single invite is recorded one way or the other, never both. In a correct implementation *talkBare* is empty in every reachable state; the model admits it precisely so the defect can be reached and then proved unreachable.

The pending-invite clock. *pendingAge* is an abstract clock for the pending set, bounded by *maxInviteAge*. It advances on poller ticks (*AgeTick*) while any invite is pending and is pinned to zero whenever no invite is pending—the invariant $(\text{dom } talkPending = \emptyset \wedge talkBare = \emptyset) \Rightarrow pendingAge = 0$ says exactly this, so a fresh invite always starts its life at age zero. Its sole purpose is to make the time-to-live path of Section 18 reachable: an invite may be expired (*ExpirePendingInvite*) only once the clock has reached its bound. The clock is deliberately global rather than per-invite; at these bounds that loses nothing the marker property depends on, and it keeps the state factor at two rather than one per user.

6 Initialization

Init

NotificationState'

```
natsConn' = natsDisconnected
kvProvisioned' = zfalse
talkSubscribed' = zfalse
session' = sessNone
toolContext' = zfalse
wallDesc' = wallDescDefault
talkDesc' = talkDescDefault
readDesc' = readDescDefault
wallDescChanged' = zfalse
talkDescChanged' = zfalse
unreadCount' = 0
talkPending' = ∅
talkBare' = ∅
talkQueued' = 0
talkConnected' = zfalse
pendingAge' = 0
queueItems' = ⟨⟩
queueKinds' = ∅
queueTexts' = ∅
currentIndex' = 0
fileExists' = zfalse
fileUnreadCount' = 0
fileItems' = ⟨⟩
barLine2' = idleMarker
barStale' = ztrue
lastNotifyResult' = notifyDropped
staleWithdrawHit' = zfalse
pendingNotify' = zfalse
notifyLost' = zfalse
inFlight' = ∅
beltPnSnapshot' = zfalse
capPnSnapshot' = zfalse
raceLost' = zfalse
```

The initial state has *session'* = *sessNone* and *natsConn'* = *natsDisconnected*: nothing has connected and nothing has been notified.

7 Session Capture

The MCP server lifespan startup captures the session reference eagerly, before the NATS connection is established and before the background poller begins. This is the key operation that prevents the *biff-8g0* bug class: by the time any NATS callback can fire, the session is guaranteed to have been captured at least once.

CaptureSession below is the case in which there is nothing to flush—no description changed while the server was starting up—so the whole operation is the synchronous assignment `_session = session` with no `await` in it, genuinely atomic. When a *pendingNotify* is recorded, capture must also flush it, and that flush does `await session.send_tool_list_changed()`: a real network call, with a real failure mode and a real gap between deciding to flush and the flush landing. Section 13 gives that case—shared with *RecaptureSession* below, since both call the same `capture_session()`—as *CaptureSessionFlushBegin*, *CaptureSessionFlushOk*, and *CaptureSessionFlushFail*.

CaptureSession
 Δ *NotificationState*

```

pendingAge' = pendingAge
session = sessNone
natsConn = natsDisconnected
pendingNotify = zfalse
session' = sessCaptured
natsConn' = natsConn
lastNotifyResult' = lastNotifyResult
wallDescChanged' = wallDescChanged
talkDescChanged' = talkDescChanged
pendingNotify' = pendingNotify
notifyLost' = notifyLost
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit

```

After *CaptureSession*, the system has *session* = *sessCaptured* with *natsConn* = *natsDisconnected*, and *EnsureConnected* can now be applied.

8 NATS Connection Operations

8.1 Ensure Connected

Lazy connection: transitions from disconnected to connected, provisioning KV and stream infrastructure. Idempotent when already connected.

The guard *session* $\neq$ *sessNone* is the biff-8g0 protection, stated here explicitly rather than derived from a state invariant (see the erratum in Section 5): the system must execute *CaptureSession* at least once before the first *EnsureConnected*. It is deliberately *not* *session* = *sessCaptured*—after a suspender's send failure (Section 13) *session* may be *sessNone* again while *natsConn* is already *natsConnected*, and *EnsureConnected* must remain applicable (idempotently) in that state without requiring a fresh capture first.

EnsureConnected
 Δ *NotificationState*

pendingAge' = *pendingAge*
session $\neq$ *sessNone* $\vee$ *natsConn* = *natsConnected*
natsConn = *natsDisconnected* $\vee$ *natsConn* = *natsConnected*
natsConn' = *natsConnected*
kvProvisioned' = *ztrue*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

8.2 Close Connection

Permanent shutdown. Clears all NATS-dependent state. The session reference is preserved by this operation—a clean shutdown does not by itself invalidate it; only a failed *send* does (Section 13).

CloseConnection
 Δ *NotificationState*

pendingAge' = *pendingAge*
natsConn' = *natsClosed*
kvProvisioned' = *zfalse*
talkSubscribed' = *zfalse*
session' = *session*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

9 Tool Context

A tool handler runs *inside* the MCP request-response cycle; the background poller and the NATS subscription callbacks run *outside* it. The component *toolContext* records which of the two the system is currently in: *ztrue* during a tool call, *zfalse* otherwise. The distinction is not decoration. It selects the notification path—belt when a request context is available, suspenders when it is not—and it gates every operation that can only occur inside a handler (*WallPost*, *TalkStart*, *NotifyBelt*) against every operation that can only occur in the background (*NotifySuspenders*, *KVWallReceive*, *NatsTalkCallback*, *PollTick*).

The earlier drafts left *toolContext* at its initial *zfalse* with no operation to raise it. The consequence was quiet but total: no reachable state had *toolContext* = *ztrue*, so the belt-path operations were enabled in no reachable state, and the guarantees stated for them (*NotifyGuarantee*, *WallNotifySuccess*) held only *vacuously*. A vacuous proof is not a proof of anything. The remedy is to model the entry into and exit from a tool handler explicitly, so that the belt path is reachable and its obligations bite.

EnterToolContext marks the start of a tool call; *LeaveToolContext* marks its end. Each toggles *toolContext* and changes nothing else, so neither can disturb any invariant. Their guards make the two mutually exclusive: one may enter only from the background, and leave only from a handler.

EnterToolContext

Δ *NotificationState*

pendingAge' = *pendingAge*
toolContext = *zfalse*
toolContext' = *ztrue*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
readDesc' = *readDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

LeaveToolContext

Δ *NotificationState*

```
pendingAge' = pendingAge
toolContext = ztrue
toolContext' = zfalse
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
session' = session
wallDesc' = wallDesc
talkDesc' = talkDesc
readDesc' = readDesc
wallDescChanged' = wallDescChanged
talkDescChanged' = talkDescChanged
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
lastNotifyResult' = lastNotifyResult
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit
```

A tool handler is therefore modelled as *EnterToolContext*, followed by the handler’s own operation on the belt path, followed by *LeaveToolContext*; background work happens between these brackets, never inside them. This is the smallest change that makes the belt path reachable without weakening any existing guard.

10 Notification Core

This is the central mechanism. Two paths send the MCP `tools/list_changed` notification. *Erratum*: earlier drafts stated the belt path “always succeeds.” Round 2’s review found it does not—*NotifyBeltSendFail* (Section 10.1)—so both paths can fail, and both are given here in the simplified, idealised form they were first understood in, before Sections 13 and 13.2 model the failure at each site precisely and Section 14 models the three sites’ non-atomicity.

10.1 Belt Path (Tool Context Available)

Inside a tool handler: notification piggybacks on the MCP response. This is the preferred path when tool context is available because it guarantees synchronous delivery within the request-response cycle—but “synchronous” describes the request-response cycle, not the call itself: `await ctx.send_notification(...)` is still a genuine `await`, and round 2’s review found it can fail with something other than the `RuntimeError`

that selects belt over suspenders in the first place—a transport error from a client that dropped mid-request. The real implementation also refreshes the stored session reference on *every* belt call, whether or not the prior reference was still good—`_session = ctx.session`, unconditionally, and only once the send has actually succeeded—so a belt call is a critical section with the same read/decide-then-await-then-write shape as the other two sites. *NotifyBeltBegin* is the read/decide step; *NotifyBelt* and *NotifyBeltSendFail* are its two possible commits.

<i>NotifyBeltBegin</i>	_____
Δ NotificationState	
<i>pendingAge'</i>	<i>pendingAge</i>
<i>toolContext</i>	<i>ztrue</i>
<i>siteBelt</i>	$\notin \text{inFlight}$
<i>inFlight'</i>	<i>inFlight</i> $\cup \{siteBelt\}$
<i>beltPnSnapshot'</i>	<i>pendingNotify</i>
<i>session'</i>	<i>session</i>
<i>natsConn'</i>	<i>natsConn</i>
<i>kvProvisioned'</i>	<i>kvProvisioned</i>
<i>talkSubscribed'</i>	<i>talkSubscribed</i>
<i>toolContext'</i>	<i>toolContext</i>
<i>readDesc'</i>	<i>readDesc</i>
<i>unreadCount'</i>	<i>unreadCount</i>
<i>talkPending'</i>	<i>talkPending</i>
<i>talkBare'</i>	<i>talkBare</i>
<i>talkQueued'</i>	<i>talkQueued</i>
<i>talkConnected'</i>	<i>talkConnected</i>
<i>wallDesc'</i>	<i>wallDesc</i>
<i>talkDesc'</i>	<i>talkDesc</i>
<i>wallDescChanged'</i>	<i>wallDescChanged</i>
<i>talkDescChanged'</i>	<i>talkDescChanged</i>
<i>queueItems'</i>	<i>queueItems</i>
<i>queueKinds'</i>	<i>queueKinds</i>
<i>queueTexts'</i>	<i>queueTexts</i>
<i>currentIndex'</i>	<i>currentIndex</i>
<i>fileExists'</i>	<i>fileExists</i>
<i>fileUnreadCount'</i>	<i>fileUnreadCount</i>
<i>fileItems'</i>	<i>fileItems</i>
<i>barLine2'</i>	<i>barLine2</i>
<i>barStale'</i>	<i>barStale</i>
<i>lastNotifyResult'</i>	<i>lastNotifyResult</i>
<i>pendingNotify'</i>	<i>pendingNotify</i>
<i>notifyLost'</i>	<i>notifyLost</i>
<i>capPnSnapshot'</i>	<i>capPnSnapshot</i>
<i>raceLost'</i>	<i>raceLost</i>
<i>staleWithdrawHit'</i>	<i>staleWithdrawHit</i>

NotifyBelt is the send-succeeds commit. It always flushes—clearing *pendingNotify* unconditionally is correct precisely because the belt call’s own send just delivered the *current* tool description, which reflects every mutation up to this point, whichever site armed *pendingNotify*. *raceLost* is Section 14’s instrument: if *pendingNotify* has changed since this call’s own *beltPnSnapshot* was taken at *NotifyBeltBegin*, some other site’s write landed in the gap, and this commit is about to overwrite it with a decision made before that write happened.

NotifyBelt

Δ *NotificationState*

```
pendingAge' = pendingAge
toolContext = ztrue
siteBelt  $\in$  inFlight
inFlight' = inFlight \ {siteBelt}
lastNotifyResult' = notifySent
session' = sessCaptured
pendingNotify' = zfalse
((pendingNotify  $\neq$  beltPnSnapshot)  $\Rightarrow$  raceLost' = ztrue)
((pendingNotify = beltPnSnapshot)  $\Rightarrow$  raceLost' = raceLost)
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
wallDescChanged' = zfalse
talkDescChanged' = zfalse
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = zfalse
notifyLost' = notifyLost
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
staleWithdrawHit' = staleWithdrawHit
```

NotifyBeltSendFail is the corrected failure commit: the send raised something other than `RuntimeError`, nothing was delivered, the session assignment line was never reached (it comes *after* the `await` in the real code, so *session* is left exactly as it was), and—the round 2 fix—*pendingNotify* is raised to record that this attempt failed. *NotifyBeltSendFailUnrecorded* is the dead twin: identical except *pendingNotify* is left untouched, which is exactly what happens if the exception handler that round 2 asks for is never written.

NotifyBeltSendFail

Δ *NotificationState*

pendingAge' = *pendingAge*
toolContext = *ztrue*
siteBelt $\in$ *inFlight*
inFlight' = *inFlight* $\setminus$ {*siteBelt*}
lastNotifyResult' = *notifyDropped*
session' = *session*
pendingNotify' = *ztrue*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
notifyLost' = *notifyLost*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

NotifyBeltSendFailUnrecorded

Δ *NotificationState*

pendingAge' = *pendingAge*
toolContext = *ztrue*
siteBelt $\in$ *inFlight*
inFlight' = *inFlight* $\setminus$ {*siteBelt*}
lastNotifyResult' = *notifyDropped*
session' = *session*
pendingNotify' = *pendingNotify*
notifyLost' = *ztrue*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

10.2 Suspenders Path (No Tool Context), idealised

Background poller: no request context, so the eagerly captured session reference is used to send the notification instead. NATS callbacks do not use this path directly—they set description-changed flags that the poller detects on its next tick. This schema is the *idealised* suspenders path, kept for continuity with the earlier drafts: it assumes the send always succeeds whenever *session* = *sessCaptured*. Section 13 replaces this assumption at the operational level with the two outcomes a real send actually has.

NotifySuspenders

Δ *NotificationState*

```
pendingAge' = pendingAge
toolContext = zfalse
session = sessCaptured
lastNotifyResult' = notifySent
session' = session
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
wallDescChanged' = zfalse
talkDescChanged' = zfalse
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = zfalse
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit
```

10.3 Total Notification

The complete notification operation is the disjunction of the two paths. Exactly one fires: *NotifyBelt* when *toolContext* = *ztrue*, *NotifySuspenders* (idealised) or its two-outcome refinement, Section 13, when *toolContext* = *zfalse*.

Erratum. The paragraph that stood here in earlier drafts claimed “no notification is ever dropped,” reasoning from the state invariant $natsConn = natsConnected \Rightarrow session = sessCaptured$ that Section 5’s erratum has since retracted. That claim is false: a suspenders send can fail independently of NATS connectivity, and root cause #1 (Section 13) is exactly the case in which the resulting drop was, until the fix given there, never recorded and never recovered. NATS callbacks still defer to the poller and wake it from napping so that the next active tick *attempts* the notification promptly; whether that attempt succeeds is the question Section 13 answers.

11 Send Flow Operations

11.1 Wall Post (Tool Handler)

The wall tool executes inside a tool handler context. It mutates the wall description, adds a display item, and fires notification via the belt path. The precondition requires tool context and a live NATS connection.

WallPost _____

$\Delta NotificationState$
 $wallText? : MSG$
 $wallKey? : SOURCEKEY$

$pendingAge' = pendingAge$
 $toolContext = ztrue$
 $natsConn = natsConnected$
 $wallDesc' \neq wallDesc$
 $wallDescChanged' = ztrue$
 $(wallKey? \notin \text{ran } queueItems \Rightarrow$
 $queueItems' = queueItems \hat{\ } \langle wallKey? \rangle \wedge$
 $queueKinds' = queueKinds \cup$
 $\{ wallKey? \mapsto itemWall \} \wedge$
 $queueTexts' = queueTexts \cup$
 $\{ wallKey? \mapsto wallText? \})$
 $(wallKey? \in \text{ran } queueItems \Rightarrow$
 $queueItems' = queueItems \wedge$
 $queueKinds' = queueKinds \wedge$
 $queueTexts' = queueTexts)$
 $session' = session$
 $lastNotifyResult' = notifySent$
 $barStale' = zfalse$
 $natsConn' = natsConn$
 $kvProvisioned' = kvProvisioned$
 $talkSubscribed' = talkSubscribed$
 $toolContext' = toolContext$
 $readDesc' = readDesc$
 $unreadCount' = unreadCount$
 $talkPending' = talkPending$
 $talkBare' = talkBare$
 $talkQueued' = talkQueued$
 $talkConnected' = talkConnected$
 $talkDesc' = talkDesc$
 $talkDescChanged' = zfalse$
 $currentIndex' = currentIndex$
 $fileExists' = fileExists$
 $fileUnreadCount' = fileUnreadCount$
 $fileItems' = fileItems$
 $barLine2' = barLine2$
 $pendingNotify' = pendingNotify$
 $notifyLost' = notifyLost$
 $inFlight' = inFlight$
 $beltPnSnapshot' = beltPnSnapshot$
 $capPnSnapshot' = capPnSnapshot$
 $raceLost' = raceLost$
 $staleWithdrawHit' = staleWithdrawHit$

11.2 Talk Start (Tool Handler)

The talk tool initiates a conversation. Runs inside tool context, and arranges for the background poller to subscribe to NATS talk notifications.

TalkStart

Δ *NotificationState*
partner? : *USER*

pendingAge' = *pendingAge*
toolContext = *ztrue*
natsConn = *natsConnected*
session' = *session*
lastNotifyResult' = *notifySent*
talkSubscribed' = *ztrue*
barStale' = *zfalse*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
wallDescChanged' = *zfalse*
talkDesc' = *talkDesc*
talkDescChanged' = *zfalse*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

11.3 External Message Deliver (CLI)

A CLI client or another MCP server delivers a message via NATS. This publishes to JetStream (reliable) and then publishes a talk notification on core NATS (fire-and-forget). No tool context exists on the sender side. The receiver's *NatsTalkCallback* handles the receive-side effects.

ExternalDeliver

Δ *NotificationState*

body? : *MSG*

toUser? : *USER*

pendingAge' = *pendingAge*
natsConn = *natsConnected*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

12 Receive Flow Operations

12.1 KV Wall Receive

Models the effects that occur when the periodic status poller, running in the MCP server coroutine context, detects that the wall KV entry has changed (typically after being woken by the NATS KV watcher). The watcher itself runs *outside* any tool handler context in a background `asyncio` task and only wakes the poller; it does *not* update the wall description, modify the display queue, or fire an MCP notification directly—sending notifications from a background task is unreliable for the same reason as NATS subscription callbacks.

After being woken, the poller performs its next active tick (every 2 seconds), reads the wall value from KV, detects any change, applies this schema to update the wall description and display queue, and then fires the notification from the MCP server's coroutine context.

The precondition *natsConn* = *natsConnected* implies *session* = *sessCaptured* (by state invariant), so the poller will have a valid session when it fires the deferred notification.

KVWallReceive

Δ *NotificationState*

wallText? : *MSG*

wallKey? : *SOURCEKEY*

pendingAge' = *pendingAge*
natsConn = *natsConnected*
toolContext = *zfalse*
wallDesc' $\neq$ *wallDesc*
wallDescChanged' = *ztrue*
(*wallKey?* $\notin$ *ran queueItems* $\Rightarrow$
 queueItems' = *queueItems* $\hat{\ } \langle \textit{wallKey?} \rangle \wedge$
 queueKinds' = *queueKinds* $\cup$
 {*wallKey?* $\mapsto$ *itemWall*} $\wedge$
 queueTexts' = *queueTexts* $\cup$
 {*wallKey?* $\mapsto$ *wallText?*})
(*wallKey?* $\in$ *ran queueItems* $\Rightarrow$
 queueItems' = *queueItems* $\wedge$
 queueKinds' = *queueKinds* $\wedge$
 queueTexts' = *queueTexts*)
lastNotifyResult' = *lastNotifyResult*
barStale' = *barStale*
session' = *session*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
talkDesc' = *talkDesc*
talkDescChanged' = *talkDescChanged*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

12.2 NATS Talk Callback

Fires when a core NATS message arrives on the talk notification subject. Runs *outside* any tool handler context in a NATS subscription callback coroutine. Mutates the talk description and adds the talk item to the display queue, but does *not* fire an MCP notification directly—sending notifications from a NATS callback is unreliable because Claude Code intermittently ignores them (different coroutine context from the MCP server).

Instead, the callback sets *talkDescChanged'* = *ztrue* and the background poller (*PollTick*) detects the change on its next tick and fires the notification from the MCP server's own coroutine context. This is the same pattern wall uses: state mutation is decoupled from notification delivery.

The precondition $talkSubscribed = ztrue$ implies $natsConn = natsConnected$ (by state invariant), which in turn implies $session = sessCaptured$. The poller will therefore have a valid session when it fires the deferred notification.

NatsTalkCallback

$\Delta NotificationState$

$senderBody? : MSG$

$senderKey? : SOURCEKEY$

$pendingAge' = pendingAge$

$talkSubscribed = ztrue$

$toolContext = zfalse$

$talkBare = \emptyset$

$talkQueued < maxQueueSize$

$talkDesc' \neq talkDescDefault$

$talkDescChanged' = ztrue$

$queueItems' = queueItems \setminus \langle senderKey? \rangle$

$queueKinds' = queueKinds \oplus$

$\{ senderKey? \mapsto itemTalk \}$

$queueTexts' = queueTexts \oplus$

$\{ senderKey? \mapsto senderBody? \}$

$currentIndex' = \#queueItems$

$lastNotifyResult' = lastNotifyResult$

$barStale' = barStale$

$session' = session$

$natsConn' = natsConn$

$kvProvisioned' = kvProvisioned$

$talkSubscribed' = talkSubscribed$

$toolContext' = toolContext$

$readDesc' = readDesc$

$unreadCount' = unreadCount$

$talkPending' = talkPending$

$talkBare' = talkBare$

$talkQueued' = talkQueued + 1$

$talkConnected' = talkConnected$

$wallDesc' = wallDesc$

$wallDescChanged' = wallDescChanged$

$fileExists' = fileExists$

$fileUnreadCount' = fileUnreadCount$

$fileItems' = fileItems$

$barLine2' = barLine2$

$pendingNotify' = pendingNotify$

$notifyLost' = notifyLost$

$inFlight' = inFlight$

$beltPnSnapshot' = beltPnSnapshot$

$capPnSnapshot' = capPnSnapshot$

$raceLost' = raceLost$

$staleWithdrawHit' = staleWithdrawHit$

12.3 Sync Unread File

Writes the display queue snapshot to the filesystem status file. Called after any state change that affects what the status bar shows.

SyncUnreadFile

Δ *NotificationState*

count? : $\mathbb{N}$

pendingAge' = *pendingAge*
count? $\leq$ *maxUnreadCount*
fileExists' = *ztrue*
fileUnreadCount' = *count?*
fileItems' = *queueItems*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

12.4 Poll Tick (Background Poller)

The 2-second background poller tick. Runs outside tool context. This schema is the *idle* case only—nothing has changed, so there is nothing to attempt. Section 13 gives the three operations that cover the remaining case, *wallDescChanged* = *ztrue* $\vee$ *talkDescChanged* = *ztrue*: a successful suspenders send, a failed one that is correctly recorded, and—kept as the now-dead root cause #1—a failed one that is silently absorbed.

PollTick

ΔNotificationState

```
pendingAge' = pendingAge
toolContext = zfalse
wallDescChanged = zfalse
talkDescChanged = zfalse
lastNotifyResult' = lastNotifyResult
barStale' = barStale
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
session' = session
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
wallDescChanged' = zfalse
talkDescChanged' = zfalse
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit
```

13 Suspenders Failure and the No-Recovery Sink

Root cause #1, “dropped `tools/list_changed` mid-session, no recovery.” A suspenders send can fail—the stored `ServerSession` may be attached to a stream the client has already torn down—independently of whether the NATS connection itself is healthy. `notify_tool_list_changed()`’s suspenders branch catches exactly this and clears the stored session reference so the poller stops hammering a dead stream:

```
if _session is not None:
    try:
        await _session.send_tool_list_changed()
    except Exception:
        logger.warning(...)
        _session = None
    return
_pending_notify = True
```

The `return` on the exception path is the defect: it skips the `_pending_notify = True` line entirely. A failed suspenders send and a session that was simply never captured end up indistinguishable—both

leave `_session = None` with nothing recorded—even though the two cases are not symmetric. The pre-session case is genuinely a first attempt with nothing yet to notify about; the suspenders-failure case has already mutated the tool description and already decided a notification is owed. Losing that distinction is what makes the drop permanent: the one flag that exists to remember an owed notification, `pendingNotify`, is never set.

`PollTickNotifyBegin` is the read/decide step: a changed flag is set, the session looks captured, so the poller commits to attempting a send. Nothing about the send’s outcome is known yet.

<i>PollTickNotifyBegin</i> Δ <i>NotificationState</i>
$pendingAge' = pendingAge$ $toolContext = zfalse$ $(wallDescChanged = ztrue \vee talkDescChanged = ztrue)$ $session = sessCaptured$ $siteSuspenders \notin inFlight$ $inFlight' = inFlight \cup \{siteSuspenders\}$ $session' = session$ $natsConn' = natsConn$ $kvProvisioned' = kvProvisioned$ $talkSubscribed' = talkSubscribed$ $toolContext' = toolContext$ $readDesc' = readDesc$ $unreadCount' = unreadCount$ $talkPending' = talkPending$ $talkBare' = talkBare$ $talkQueued' = talkQueued$ $talkConnected' = talkConnected$ $wallDesc' = wallDesc$ $talkDesc' = talkDesc$ $wallDescChanged' = wallDescChanged$ $talkDescChanged' = talkDescChanged$ $queueItems' = queueItems$ $queueKinds' = queueKinds$ $queueTexts' = queueTexts$ $currentIndex' = currentIndex$ $fileExists' = fileExists$ $fileUnreadCount' = fileUnreadCount$ $fileItems' = fileItems$ $barLine2' = barLine2$ $barStale' = barStale$ $lastNotifyResult' = lastNotifyResult$ $pendingNotify' = pendingNotify$ $notifyLost' = notifyLost$ $beltPnSnapshot' = beltPnSnapshot$ $capPnSnapshot' = capPnSnapshot$ $raceLost' = raceLost$ $staleWithdrawHit' = staleWithdrawHit$

`PollTickNotifyOk` commits a successful send: releases `siteSuspenders` from `inFlight`, clears the changed flags, and does not touch `pendingNotify`—the suspenders path never has anything of its own to flush, only the current tick’s own delivery to report.

PollTickNotifyOk

Δ *NotificationState*

pendingAge' = *pendingAge*
toolContext = *zfalse*
siteSuspenders $\in$ *inFlight*
inFlight' = *inFlight* $\setminus$ {*siteSuspenders*}
lastNotifyResult' = *notifySent*
barStale' = *zfalse*
session' = *session*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *zfalse*
talkDescChanged' = *zfalse*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

PollTickNotifyFail is the corrected failure commit: the send fails, the session reference is cleared exactly as the code above clears it, the changed flag that triggered the attempt is left set (the client was not, in fact, told anything), and—the fix—*pendingNotify* is raised to record the debt.

PollTickNotifyFail

Δ *NotificationState*

pendingAge' = *pendingAge*
toolContext = *zfalse*
siteSuspenders $\in$ *inFlight*
inFlight' = *inFlight* $\setminus$ {*siteSuspenders*}
lastNotifyResult' = *notifyDropped*
barStale' = *barStale*
session' = *sessNone*
pendingNotify' = *ztrue*
notifyLost' = *notifyLost*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

PollTickNotifyFailUnrecorded is root cause #1 itself, kept as the negative result it now is: identical precondition and identical *session'* = *sessNone*, but *pendingNotify* is left exactly as the **return** in the code above leaves it—untouched. *notifyLost'* = *ztrue* records that this has happened, the same device *staleWithdrawHit* uses for the talk channel's reordering defect.

<i>PollTickNotifyFailUnrecorded</i> <i>ΔNotificationState</i>
<i>pendingAge'</i> = <i>pendingAge</i>
<i>toolContext</i> = <i>zfalse</i>
<i>siteSuspenders</i> ∈ <i>inFlight</i>
<i>inFlight'</i> = <i>inFlight</i> \ { <i>siteSuspenders</i> }
<i>lastNotifyResult'</i> = <i>notifyDropped</i>
<i>barStale'</i> = <i>barStale</i>
<i>session'</i> = <i>sessNone</i>
<i>pendingNotify'</i> = <i>pendingNotify</i>
<i>notifyLost'</i> = <i>ztrue</i>
<i>natsConn'</i> = <i>natsConn</i>
<i>kvProvisioned'</i> = <i>kvProvisioned</i>
<i>talkSubscribed'</i> = <i>talkSubscribed</i>
<i>toolContext'</i> = <i>toolContext</i>
<i>readDesc'</i> = <i>readDesc</i>
<i>unreadCount'</i> = <i>unreadCount</i>
<i>talkPending'</i> = <i>talkPending</i>
<i>talkBare'</i> = <i>talkBare</i>
<i>talkQueued'</i> = <i>talkQueued</i>
<i>talkConnected'</i> = <i>talkConnected</i>
<i>wallDesc'</i> = <i>wallDesc</i>
<i>talkDesc'</i> = <i>talkDesc</i>
<i>wallDescChanged'</i> = <i>wallDescChanged</i>
<i>talkDescChanged'</i> = <i>talkDescChanged</i>
<i>queueItems'</i> = <i>queueItems</i>
<i>queueKinds'</i> = <i>queueKinds</i>
<i>queueTexts'</i> = <i>queueTexts</i>
<i>currentIndex'</i> = <i>currentIndex</i>
<i>fileExists'</i> = <i>fileExists</i>
<i>fileUnreadCount'</i> = <i>fileUnreadCount</i>
<i>fileItems'</i> = <i>fileItems</i>
<i>barLine2'</i> = <i>barLine2</i>
<i>beltPnSnapshot'</i> = <i>beltPnSnapshot</i>
<i>capPnSnapshot'</i> = <i>capPnSnapshot</i>
<i>raceLost'</i> = <i>raceLost</i>
<i>staleWithdrawHit'</i> = <i>staleWithdrawHit</i>

13.1 Recovery: a mid-session reconnect

CaptureSession (Section 7) is guarded to the server’s startup window—*natsConn* = *natsDisconnected*—and cannot fire again once NATS is up. But the real `_SessionCaptureMiddleware` runs `capture_session()` on *every* client `initialize`, not only the first: a client that reconnects mid-session (a fresh MCP transport, the same NATS connection underneath) re-arrives here. *RecaptureSession* below is the no-flush case of that second call, exactly mirroring *CaptureSession*’s own no-flush case: nothing was pending, so the assignment is synchronous and atomic.

RecaptureSession
 Δ *NotificationState*

```

pendingAge' = pendingAge
session = sessNone
natsConn = natsConnected
pendingNotify = zfalse
session' = sessCaptured
natsConn' = natsConn
lastNotifyResult' = lastNotifyResult
wallDescChanged' = wallDescChanged
talkDescChanged' = talkDescChanged
pendingNotify' = pendingNotify
notifyLost' = notifyLost
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit

```

13.2 Capture/Reconnect Flush

Both *CaptureSession* and *RecaptureSession* call the same `capture_session()`, and when *pendingNotify* = *ztrue* that function does `await session.send_tool_list_changed()` before returning—the same shape of critical section as the belt and suspenders paths, and subject to the same two rounds of findings: round 1’s missing re-arm on failure, and round 2’s non-atomicity. *CaptureSessionFlushBegin* fires from either entry context (the disjunction on *natsConn* is exactly *CaptureSession*’s and *RecaptureSession*’s two guards); *CaptureSessionFlushOk* and *CaptureSessionFlushFail* are its two commits, sharing one critical section and one *capPnSnapshot* because they are, in the running system, the same function.

CaptureSessionFlushBegin

Δ *NotificationState*

pendingAge' = *pendingAge*
session = *sessNone*
(*natsConn* = *natsDisconnected* $\vee$ *natsConn* = *natsConnected*)
pendingNotify = *ztrue*
siteCapture $\notin$ *inFlight*
inFlight' = *inFlight* $\cup$ {*siteCapture*}
capPnSnapshot' = *pendingNotify*
session' = *session*
natsConn' = *natsConn*
lastNotifyResult' = *lastNotifyResult*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
beltPnSnapshot' = *beltPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

CaptureSessionFlushOk: the flush send succeeds, delivering whatever the tool descriptions currently hold. *raceLost* triggers on the same stale-snapshot test as *NotifyBelt*'s.

CaptureSessionFlushOk

Δ *NotificationState*

```
pendingAge' = pendingAge
session = sessNone
siteCapture ∈ inFlight
inFlight' = inFlight \ {siteCapture}
session' = sessCaptured
natsConn' = natsConn
lastNotifyResult' = notifySent
wallDescChanged' = zfalse
talkDescChanged' = zfalse
pendingNotify' = zfalse
((pendingNotify ≠ capPnSnapshot) ⇒ raceLost' = ztrue)
((pendingNotify = capPnSnapshot) ⇒ raceLost' = raceLost)
notifyLost' = notifyLost
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
beltPnSnapshot' = beltPnSnapshot
staleWithdrawHit' = staleWithdrawHit
```

CaptureSessionFlushFail is round 2's fix at this site: the flush send fails, the session reference reverts to *sessNone* (the real code's exception handler clears `_session` here too), and *pendingNotify* is re-armed rather than left cleared. *CaptureSessionFlushFailUnrecorded* is the dead twin: the real code's `_pending_notify = False` runs *before* the `await`, so an exception handler that does not explicitly re-arm leaves it cleared—exactly the shape of round 1's suspenders defect, at a different site.

CaptureSessionFlushFail

Δ *NotificationState*

pendingAge' = *pendingAge*
session = *sessNone*
siteCapture $\in$ *inFlight*
inFlight' = *inFlight* $\setminus$ {*siteCapture*}
session' = *sessNone*
natsConn' = *natsConn*
lastNotifyResult' = *notifyDropped*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
pendingNotify' = *ztrue*
notifyLost' = *notifyLost*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
beltPnSnapshot' = *beltPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

CaptureSessionFlushFailUnrecorded
 Δ *NotificationState*

```

pendingAge' = pendingAge
session = sessNone
siteCapture  $\in$  inFlight
inFlight' = inFlight  $\setminus$  {siteCapture}
session' = sessNone
natsConn' = natsConn
lastNotifyResult' = notifyDropped
wallDescChanged' = wallDescChanged
talkDescChanged' = talkDescChanged
pendingNotify' = zfalse
notifyLost' = ztrue
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
beltPnSnapshot' = beltPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit

```

Root cause #1's second consequence, restated precisely now that the flush is its own critical section: a reconnect that arrives after *PollTickNotifyFailUnrecorded* (round 1's defect, *pendingNotify* still *zfalse*) finds nothing to flush and takes the no-flush *RecaptureSession* path above—restoring *session* to *sessCaptured* but delivering nothing, because nothing recorded that anything was owed. Once round 1's fix is in place, *pendingNotify* = *ztrue* survives to the reconnect, *CaptureSessionFlushBegin* fires instead, and the missed notification is delivered as part of the flush—*unless* round 2's race clobbers it first, which Section 14 takes up next.

13.3 The counterexample

Deleting *notifyLost* = *zfalse* from *NotificationState* (Section 5) and setting the ProB goal to *notifyLost* = *ztrue* $\wedge$ *wallDescChanged* = *ztrue* $\wedge$ *talkDescChanged* = *zfalse*—isolating the wall channel so the trace is not cluttered by unrelated talk transitions—finds the four-step trace:

```

probccli docs/notification.tex -model_check -noltl -coverage \
  -goal "notifyLost = ztrue & wallDescChanged = ztrue
        & talkDescChanged = zfalse" \
  -p DEFAULT_SETSIZE 2 \
  -card USER 1 -card SOURCEKEY 1 -card SESSIONKEY 1 \
  -card QSLLOT 1 -card USLOT 1 -card ASLOT 1 -card PSLOT 1 \
  -p MAX_INITIALISATIONS 16 -p MAX_OPERATIONS 8000

```

```

Found GOAL in state id 1591
Model checking time: 291 ms (304 ms walltime)
States analysed: 423
Transitions fired: 4630
*** COUNTER EXAMPLE FOUND ***
*** TRACE (length=6):
  1: SETUP_CONSTANTS(...)
  2: INITIALISATION(..., natsConn=natsDisconnected, notifyLost=zfalse,
    pendingNotify=zfalse, session=sessNone, ...)
  3: CaptureSession(session=sessCaptured)
  4: EnsureConnected(kvProvisioned=ztrue,natsConn=natsConnected)
  5: KVWallReceive(SOURCEKEY1,MSG2)
  6: PollTickNotifyFailUnrecorded

```

Step 5 mutates the wall description and sets $wallDescChanged' = ztrue$; nothing has been sent yet, that is the poller's job on its next tick. Step 6 is the suspenders send failing on that very next tick: $session' = sessNone$, $wallDescChanged$ is left $ztrue$ (the client was not told), $pendingNotify' = pendingNotify = zfalse$ (the drop is not recorded), $notifyLost' = ztrue$. The resulting state is exactly the one the task asks for: $wallDescChanged = ztrue$ (activity is server-side correct and waiting), $session = sessNone$ (the suspenders path is dead), $pendingNotify = zfalse$ (nothing will tell the next capture to flush), and $natsConn = natsConnected$ throughout (the connection itself was never at fault).

RecaptureSession is enabled from this state—its guard $session = sessNone \wedge natsConn = natsConnected \wedge pendingNotify = zfalse$ holds (round 1's defect never sets $pendingNotify$)—and confirms the second half of the finding without needing a further ProB search: by inspection of *RecaptureSession*'s guard and body (Section 13.2), firing it sets $session' = sessCaptured$ while leaving $wallDescChanged'$, $talkDescChanged'$, and $lastNotifyResult'$ all unchanged. The reconnect does not recover the missed notification, because nothing recorded that one was owed.

```

probccli docs/notification.tex -model_check -noltl -coverage \
  -p DEFAULT_SETSIZE 2 \
  -card USER 1 -card SOURCEKEY 1 -card SESSIONKEY 1 \
  -card QSLLOT 1 -card USLOT 1 -card ASLOT 1 -card PSLLOT 1 \
  -p MAX_INITIALISATIONS 16 -p MAX_OPERATIONS 8000

```

What this does and does not establish. Round 1's own exhaustive run at this scope—1,086,889 states, *invariant_violated*:0, *deadlocked*:0, *open*:0, *PollTickNotifyFailUnrecorded* and *NotifyBeltStale* (the earlier, single-site name for the belt defect, since retired—see Section 10.1) reported uncovered—was computed before round 2 added *CaptureSessionFlushBegin*, *NotifyBeltBegin*, *PollTickNotifyBegin*, and the *inFlight*/snapshot machinery. Re-running the same exhaustive, no-goal check against the extended schema at the same floor scope did not terminate in a reasonable time or memory budget—the reachable space, thirty-odd talk/wall/read marker components combined with the new critical-section splitting, grew past 6.5 million states and was still climbing after twenty minutes when the run was stopped. None of that growth comes from the property being checked: $notifyLost = zfalse$ depends on five variables ($pendingNotify$, $inFlight$, the two snapshots, itself), not on the talk/wall/read machinery it happened to be checked alongside. *notification-race.tex*, cited in full in Section 14, restates the property (and $raceLost = zfalse$ alongside it) against exactly those five variables in their own eleven-operation machine, and both hold exhaustively there in 23 states and 33 ms, with *PollTickNotifyFailUnrecorded*, *NotifyBeltSendFailUnrecorded*, and *CaptureSessionFlushFailUnrecorded* all reported uncovered—the same proof shape as *WithdrawStale* (Section 19), and the authority for the claim that follows.

This is a *safety* property: it says the failure-accounting obligation is never skipped, in any reachable state. It is deliberately not phrased as the CTL liveness formula $AG((wallDescChanged = ztrue \vee talkDescChanged = ztrue) \Rightarrow EF(wallDescChanged = zfalse \wedge talkDescChanged = zfalse))$ that closes the talk channel's own defect in Section 19, because that formula would report TRUE in *both* the buggy and the fixed model here: even though *NotifyBelt* can now fail too, its success commit remains enabled whenever $toolContext = ztrue$ and a belt call is in flight, so ProB's existential path quantifier EF always finds a recovering trace through some future, unrelated tool call succeeding, whether or not the suspenders bookkeeping was correct. That is not a false proof, but it is not the property this bug is about, and stating it as though it were would overclaim what the check shows—the *fuzz* discipline is to report exactly what a green check establishes, not what would be reassuring for it to establish. The talk

channel's liveness proof works because *ExpirePendingInvite* is driven by a clock (*AgeTick*, *pendingAge*) guaranteed to advance on its own; nothing plays that role for the suspenders path once *session* is lost. The operations that restore it—*CaptureSession*, *RecaptureSession*, *CaptureSessionFlushOk*, *NotifyBelt*—are all environment-triggered (server startup, a client reconnect, a fresh tool call) rather than time-driven, so no purely background, poller-only sequence of transitions ever clears a stranded changed flag. That structural fact is visible by inspection of the schema set above: no operation whose guard depends only on *toolContext* = *zfalse* sets *session'* = *sessCaptured* from *session* = *sessNone*. The safety property *notifyLost* = *zfalse* is the one this specification actually proves, and it is the one the fix needs: once every suspenders failure is recorded, the next capture point—whichever arrives first—discharges it, by inspection of *CaptureSession*'s, *RecaptureSession*'s, and *CaptureSessionFlushOk*'s guards.

14 The Race: Three Critical Sections, One Shared Flag

Every operation above that writes *pendingNotify* was, until this section, modelled as a single atomic step: read the old value, decide the new one, send, write—all in one transition. The real code is not atomic there. Each of the three sites does `read-or-decide; await send(...); write`, and Python's cooperative scheduler can run any other coroutine during the `await`. *CriticalSite* and *inFlight* (Section 5) exist to make that gap a reachable state: *Begin* at each site records that the read/decide has happened and the write is still outstanding; *Commit* performs the write. Between a site's *Begin* and its *Commit*, any *other* site's *Begin* can fire—unless something forbids it.

The race, reproduced. Take $\#inFlight \leq 1$ out of *NotificationState* (Section 5) and set the ProB goal to *raceLost* = *ztrue*:

```
probccli docs/notification.tex -model_check -noltl -coverage \
  -goal "raceLost = ztrue & natsConn = natsConnected
        & pendingNotify = zfalse" \
  -p DEFAULT_SETSIZE 2 \
  -card USER 1 -card SOURCEKEY 1 -card SESSIONKEY 1 \
  -card QSLLOT 1 -card USLOT 1 -card ASLOT 1 -card PSLOT 1 \
  -p MAX_INITIALISATIONS 16 -p MAX_OPERATIONS 8000
```

Found GOAL in state id 11897

States analysed: 5985 Transitions fired: 87919

*** COUNTER EXAMPLE FOUND ***

*** TRACE (length=12):

- 1: SETUP_CONSTANTS(...)
- 2: INITIALISATION(..., session=sessNone, pendingNotify=zfalse, inFlight={}, raceLost=zfalse, ...)
- 3: EnterToolContext
- 4: NotifyBeltBegin
- 5: NotifyBeltSendFail
- 6: SyncUnreadFile(0)
- 7: StatusBarRefresh(barLine2=MSG2)
- 8: CaptureSessionFlushBegin
- 9: NotifyBeltBegin
- 10: CaptureSessionFlushOk(inFlight={siteBelt}, lastNotifyResult=notifySent, pendingNotify=zfalse, session=sessCaptured)
- 11: EnsureConnected(kvProvisioned=ztrue, natsConn=natsConnected)
- 12: NotifyBelt(barStale=zfalse, inFlight={}, raceLost=ztrue)

ProB's shortest witness is a belt/capture collision, not the belt/suspenders one sketched informally above—but it is the same mechanism, and it is worth walking precisely rather than re-describing the informal sketch as though it were what was found. Step 4–5: a first belt call's send fails; its commit re-arms *pendingNotify' = ztrue*. Step 8: capture's flush begins, reading that re-armed *ztrue* and snapshotting *capPnSnapshot = ztrue*. Step 9: *before capture commits*, a *second* belt call begins—only reachable because the mutex invariant is absent, so *siteBelt* can re-enter *inFlight* once its first occupant released it at step 5—and this second call snapshots *beltPnSnapshot = ztrue* too, reading the same still-outstanding debt. Step 10: capture's flush succeeds first, correctly delivering the notification and clearing

pendingNotify' = *zfalse*. Step 12: the second belt call's commit, made on a decision formed back at step 9 when *pendingNotify* was still *ztrue*, now finds *pendingNotify* = *zfalse* $\neq$ *beltPnSnapshot* = *ztrue*—the value it decided against has already changed—and *raceLost'* = *ztrue* fires.

In this particular interleaving the final value of *pendingNotify* happens to still be correct (*zfalse*, and it was genuinely delivered—by capture, at step 10): the harm here is redundant, order-dependent work, not a lost debt. *raceLost* is deliberately the more conservative check—any commit whose snapshot is stale at commit time, not only ones that provably corrupt the final value—because the *class* of behaviour is unsafe regardless of whether a given interleaving happens to land safely: change step 10's outcome to a failure instead of a success (a second, independent send failure, fully reachable by the same construction) and the identical stale-snapshot commit at step 12 clobbers a live re-arm exactly as the belt/suspenders sketch described. *raceLost* = *ztrue* is the witness for that whole class, not only for its most damaging member.

The mutex. $\#inFlight \leq 1$ is the Z-level statement of a lock held for the duration of each site's critical section: at most one of *siteSuspenders*, *siteBelt*, *siteCapture* may have a write outstanding at any reachable state, regardless of which two sites collide. Restoring it makes every *Begin* whose site is not already the sole occupant of *inFlight* infeasible while another site is mid-flight—not by adding a new guard to each *Begin* schema, but because any reachable state with two sites in *inFlight* would violate the promoted invariant, so ProB never visits one. Under the mutex, step 9 above cannot happen between steps 8 and 10: the second *NotifyBeltBegin*'s *siteBelt* $\notin$ *inFlight* guard is satisfiable (belt's own first occupancy already released at step 5), but the post-state *inFlight'* = {*siteCapture*, *siteBelt*} has $\#inFlight' = 2$, contradicting the invariant, so the operation is enabled in no reachable state while *siteCapture* is still held. The second belt call must wait for capture's commit (step 10) to release the lock first; by the time its *Begin* can fire, its snapshot *beltPnSnapshot'* = *pendingNotify* reads the value *capture just delivered*, *zfalse*, so its own decision is now “nothing to flush”—it commits a genuine no-op rather than a stale, redundant clear, and *raceLost* never fires. The same argument applies verbatim to a suspenders/belt or suspenders/capture pairing: the mutex forbids *any* two of the three sites having a write outstanding at once, so whichever pair a given execution would have collided on, the second to attempt *Begin* waits and reads the first's committed result instead of a value it will overwrite.

Running that check—*notifyLost* = *zfalse* and $\#inFlight \leq 1$ both restored—against the full *Notification.State* at this floor scope is exactly the run that did not terminate (Section 13's “What this does and does not establish”): 6.5 million states and climbing after twenty minutes, stopped rather than left to find out how much further it would grow. The reachable-space explosion is the talk/wall/read marker machinery's, not the race's—nothing about $\#inFlight \leq 1$ or *raceLost* depends on *talkPending*, *queueItems*, or any of the other two dozen components *Notification.State* carries for unrelated channels. **notification-race.tex** isolates the five variables the mutex and the race actually touch as their own machine and checks the identical property there:

```
probccli docs/notification-race.tex -model_check -noltl -coverage \
  -p DEFAULT_SETSIZE 2 -p MAX_OPERATIONS 2000
```

23 states, complete, in 33 ms: *invariant_violated*:0, *deadlocked*:0, *open*:0. *PollTickNotifyFailUnrecorded*, *NotifyBeltSendFailUnrecorded*, and *CaptureSessionFlushFailUnrecorded* are reported uncovered there (round 1's finding, now proved at all three sites in the same run); no operation is reported uncovered *because of* the mutex—every *Begin/Commit* pair is exercised at least once, just never with a second site's *Begin* landing while the first is still mid-flight. *raceLost* = *zfalse* and *notifyLost* = *zfalse* both hold across the whole reachable space of that isolated machine. The schemas there are verbatim restrictions of the ones above—*SusBegin/SusCommitSet* to *PollTickNotifyBegin/Fail*, *BeltBegin/BeltCommitClear/BeltCommitSet* to *NotifyBeltBegin/NotifyBelt/NotifyBeltSendFail*, and *CapBegin/CapCommitClear/CapCommitSet* to *CaptureSessionFlushBegin/Ok/Fail*—so the isolated proof transfers: the omitted variables cannot make the mutex insufficient, because none of them appear in any guard or effect that touches *pendingNotify*, *inFlight*, or the two snapshots.

Locking scope, for the implementation. The mutex proved above is a single lock shared by all three sites—not three separate locks, because the race is precisely between *different* sites, and three independent locks would each protect their own site from itself while leaving every cross-site interleaving open. A single `asyncio.Lock()` at module scope in `_descriptions.py`, held across exactly the following region at each site, is what the proof requires:

- **Suspenders** (`notify_tool_list_changed()`, suspenders branch): acquire before the `if _session is not None`: read; hold across `await _session.send_tool_list_changed()`; release only after *both* `_session` and `_pending_notify` have been written in the `except` branch (or, on success, after the function returns). The pre-session branch (`_pending_notify = True`) is a plain synchronous write with no `await` in it and does not need the lock on its own account, but taking it anyway is harmless and keeps one acquisition point per call.
- **Belt** (`notify_tool_list_changed()`, belt branch): acquire before `ctx = get_context()`; hold across `await ctx.send_notification(...)`; release only after `_session = ctx.session` on success, or after the (new, round 2) `_pending_notify = True` re-arm on a caught `non-RuntimeError` failure. Do *not* release between the send and the session/pending-notify write—that gap is exactly what *beltPnSnapshot* models, and releasing early reopens the race the proof just closed.
- **Capture/reconnect flush** (`capture_session()`): acquire before the `if not _pending_notify`: `return` read (a session that has nothing to flush still needs the lock to observe a consistent *pendingNotify*, matching *CaptureSessionFlushBegin*’s read of it); hold across `await session.send_tool_list_changed()`; release only after `_pending_notify` and `_session` are both settled in the `except` branch (round 2: set `_pending_notify = True` there, matching *CaptureSessionFlushFail*), or after the function returns on success. `_session = session`, the unconditional first line of `capture_session()`, happens before the *pendingNotify* check and is not itself part of this critical section—it has no failure mode and nothing else reads or writes *pendingNotify* around it.

The scope is the same shape at all three sites: *acquire before the last read of `_pending_notify`, release after the last write of `_pending_notify` (and, where the site also owns it, `_session`)—with the `await in between` held under the lock throughout*. A lock acquired only around the write, or released before the write, does not close the race: *beltPnSnapshot* and *capPnSnapshot* are taken at *Begin* precisely because the proof depends on the *read* being inside the same held region as the eventual write, not just the write being atomic on its own.

14.1 Status Bar Refresh

Claude Code polls the status line command on its own schedule. Reads the filesystem status file and updates the display. This is the slow path—no MCP involvement.

StatusBarRefresh

Δ *NotificationState*

```
pendingAge' = pendingAge
(fileExists = ztrue  $\wedge$ 
  fileItems  $\neq$   $\langle \rangle$   $\Rightarrow$ 
  barLine2'  $\neq$  idleMarker)
(fileExists = zfalse  $\Rightarrow$  barLine2' = idleMarker)
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
session' = session
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
wallDescChanged' = wallDescChanged
talkDescChanged' = talkDescChanged
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barStale' = barStale
lastNotifyResult' = lastNotifyResult
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit
```

15 Display Queue Operations

15.1 Advance If Due

Rotation: wall items cycle (persistent), talk items are ephemeral (removed after one display turn). The *currentIndex* + 1 expression indexes into the 1-based sequence.

Talk items at the current position are removed (ephemeral). Wall items cycle: the cursor advances, but the item stays in the queue. With a single wall item, the cursor stays at zero.

AdvanceTalk

Δ *NotificationState*

pendingAge' = *pendingAge*
queueItems $\neq \langle \rangle$
queueKinds(*queueItems*(*currentIndex* + 1)) = *itemTalk*
#queueItems' = *#queueItems* - 1
queueKinds' =
 {*queueItems*(*currentIndex* + 1)} $\triangleleft$ *queueKinds*
queueTexts' =
 {*queueItems*(*currentIndex* + 1)} $\triangleleft$ *queueTexts*
ran queueItems' = *ran queueItems* \
 {*queueItems*(*currentIndex* + 1)}
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
readDesc' = *readDesc*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

AdvanceWall
 $\Delta NotificationState$

```

pendingAge' = pendingAge
queueItems ≠ ⟨⟩
queueKinds(queueItems(currentIndex + 1)) = itemWall
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
(#queueItems > 1 ⇒
  currentIndex' = (currentIndex + 1) mod
    #queueItems)
(#queueItems = 1 ⇒ currentIndex' = 0)
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
session' = session
toolContext' = toolContext
readDesc' = readDesc
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
wallDesc' = wallDesc
talkDesc' = talkDesc
wallDescChanged' = wallDescChanged
talkDescChanged' = talkDescChanged
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
lastNotifyResult' = lastNotifyResult
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit

```

16 Tool-Description Marker Channels

The MCP server surfaces incoming activity by mutating a tool's *description* and firing `tools/list_changed`: the agent notices the changed text on its next turn. Two such channels are modelled here. The `read_messages` channel is the reference: it is known to behave correctly. The `talk` channel is the one under examination.

The governing property, developed in Section 17, is that for each channel the marker is present exactly when the underlying activity exists:

$$desc \neq default \Leftrightarrow activity.$$

This section gives the operations of both channels. It deliberately gives them first *without* the reconciling transitions that make the property hold, so that Section 17 can exhibit the defect as a reachable state before Section 18 repairs it.

16.1 Read Channel (the self-clearing reference)

Mail arrives and raises the unread count; the description gains its marker. Reading mail returns the count to zero and the description reverts. Because the description is driven by a count that *decreases*, this channel clears itself: there is no way to leave the marker set once the count reaches zero.

MailArrive

Δ *NotificationState*

pendingAge' = *pendingAge*
unreadCount < *maxUnreadCount*
unreadCount' = *unreadCount* + 1
readDesc' $\neq$ *readDescDefault*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

ReadMail

Δ *NotificationState*

pendingAge' = *pendingAge*
unreadCount > 0
unreadCount' = 0
readDesc' = *readDescDefault*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
wallDesc' = *wallDesc*
talkDesc' = *talkDesc*
wallDescChanged' = *wallDescChanged*
talkDescChanged' = *talkDescChanged*
talkPending' = *talkPending*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

16.2 Talk Channel (invite, message, drain)

An invite arrives and is recorded in *talkPending*, retaining the inviter's session key; the description gains its [TALK] marker. A talk message arrives and raises *talkQueued*. Both set the marker.

TalkInviteArrive fires when the invite either supersedes one already recorded for that user (*inviter*? $\in \text{dom } \textit{talkPending}$, no growth) or there is room for a new entry ($\# \textit{talkPending} + \# \textit{talkBare} < \textit{maxPending}$). This is the exact analogue of *ReceiveNotification*'s $\textit{queueLen} < \textit{maxQueueLen}$ guard in *talk.tex*: it keeps the pending set within its bound. When a *new* inviter arrives and the set is already full, *TalkInviteArriveOverflow* takes over, evicting one existing entry before inserting—the drop-to-make-room counterpart of *ReceiveOverflow*.

TalkInviteArrive

Δ *NotificationState*

inviter? : *USER*

inviterKey? : *SESSIONKEY*

pendingAge' = *pendingAge*

natsConn = *natsConnected*

inviter? $\notin$ *talkBare*

(*inviter?* $\in$ dom *talkPending* $\vee$ #*talkPending* + #*talkBare* < *maxPending*)

talkPending' = *talkPending* $\oplus$ {*inviter?* $\mapsto$ *inviterKey?*}

talkDesc' $\neq$ *talkDescDefault*

talkDescChanged' = *ztrue*

natsConn' = *natsConn*

kvProvisioned' = *kvProvisioned*

talkSubscribed' = *talkSubscribed*

session' = *session*

toolContext' = *toolContext*

wallDesc' = *wallDesc*

readDesc' = *readDesc*

wallDescChanged' = *wallDescChanged*

unreadCount' = *unreadCount*

talkBare' = *talkBare*

talkQueued' = *talkQueued*

talkConnected' = *talkConnected*

queueItems' = *queueItems*

queueKinds' = *queueKinds*

queueTexts' = *queueTexts*

currentIndex' = *currentIndex*

fileExists' = *fileExists*

fileUnreadCount' = *fileUnreadCount*

fileItems' = *fileItems*

barLine2' = *barLine2*

barStale' = *barStale*

lastNotifyResult' = *lastNotifyResult*

pendingNotify' = *pendingNotify*

notifyLost' = *notifyLost*

inFlight' = *inFlight*

beltPnSnapshot' = *beltPnSnapshot*

capPnSnapshot' = *capPnSnapshot*

raceLost' = *raceLost*

staleWithdrawHit' = *staleWithdrawHit*

The pending set is bounded by *maxPending* (#*PSLOT*), exactly as the message queue is bounded by *maxQueueSize*. When a new inviter arrives and the set is full, *TalkInviteArriveOverflow* evicts one existing entry—the oldest by arrival in the implementation—and inserts the new one, so the bound is held and no stream of forged inviters can exhaust memory. The model records the eviction only as the removal of *some* pending entry *evicted?*: *talkPending* is an unordered function, so arrival order is not expressible here, and “oldest by arrival” is the obligation the implementation discharges. What the model proves is the invariant—#*talkPending* + #*talkBare* $\leq$ *maxPending* across the whole reachable space—not the choice of victim. This is the pending-set mirror of *ReceiveOverflow* in *talk.tex*, where a full queue drops its head before appending.

TalkInviteArriveOverflow

Δ *NotificationState*

inviter? : *USER*

inviterKey? : *SESSIONKEY*

evicted? : *USER*

pendingAge' = *pendingAge*

natsConn = *natsConnected*

inviter? $\notin$ *talkBare*

inviter? $\notin$ *dom talkPending*

$\#talkPending + \#talkBare = maxPending$

evicted? $\in$ *dom talkPending*

talkPending' = ($\{evicted?\} \triangleleft talkPending$) $\oplus$ $\{inviter? \mapsto inviterKey?\}$

talkDesc' $\neq$ *talkDescDefault*

talkDescChanged' = *ztrue*

natsConn' = *natsConn*

kvProvisioned' = *kvProvisioned*

talkSubscribed' = *talkSubscribed*

session' = *session*

toolContext' = *toolContext*

wallDesc' = *wallDesc*

readDesc' = *readDesc*

wallDescChanged' = *wallDescChanged*

unreadCount' = *unreadCount*

talkBare' = *talkBare*

talkQueued' = *talkQueued*

talkConnected' = *talkConnected*

queueItems' = *queueItems*

queueKinds' = *queueKinds*

queueTexts' = *queueTexts*

currentIndex' = *currentIndex*

fileExists' = *fileExists*

fileUnreadCount' = *fileUnreadCount*

fileItems' = *fileItems*

barLine2' = *barLine2*

barStale' = *barStale*

lastNotifyResult' = *lastNotifyResult*

pendingNotify' = *pendingNotify*

notifyLost' = *notifyLost*

inFlight' = *inFlight*

beltPnSnapshot' = *beltPnSnapshot*

capPnSnapshot' = *capPnSnapshot*

raceLost' = *raceLost*

staleWithdrawHit' = *staleWithdrawHit*

TalkMessageArrive

Δ *NotificationState*

```
pendingAge' = pendingAge
natsConn = natsConnected
talkQueued < maxQueueSize
talkQueued' = talkQueued + 1
talkDesc' ≠ talkDescDefault
talkDescChanged' = ztrue
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
session' = session
toolContext' = toolContext
wallDesc' = wallDesc
readDesc' = readDesc
wallDescChanged' = wallDescChanged
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkConnected' = talkConnected
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
lastNotifyResult' = lastNotifyResult
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit
```

The defect (biff-9la, observed live). The drain that ends a talk—the aggregate of accepting, exchanging, and hanging up—empties every source of talk activity, but leaves *talkDesc* untouched. Nothing reverts the marker; there is no withdrawal frame and no expiry. *TalkDrainStale* models exactly this: it clears *talkPending*, *talkBare*, *talkQueued* and *talkConnected*, yet preserves *talkDesc*.

TalkDrainState

Δ *NotificationState*

```
pendingAge' = 0
(dom talkPending  $\neq \emptyset \vee$  talkBare  $\neq \emptyset \vee$  talkQueued  $> 0 \vee$  talkConnected = ztrue)
talkPending' =  $\emptyset$ 
talkBare' =  $\emptyset$ 
talkQueued' = 0
talkConnected' = zfalse
talkDesc' = talkDesc
talkDescChanged' = talkDescChanged
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
session' = session
toolContext' = toolContext
wallDesc' = wallDesc
readDesc' = readDesc
wallDescChanged' = wallDescChanged
unreadCount' = unreadCount
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
lastNotifyResult' = lastNotifyResult
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit
```

The coupled defect. A second, independent fault records an invite *without* its session key—the tty is dropped on the way into the pending representation. *TalkInviteArriveBare* puts the inviter into *talkBare* and raises the marker; the resulting accept hint reads a bare **talk @user**, which fails at the prompt (“Talk needs a specific session”).

TalkInviteArriveBare

Δ *NotificationState*

inviter? : *USER*

pendingAge' = *pendingAge*
natsConn = *natsConnected*
inviter? $\notin$ dom *talkPending*
talkBare' = *talkBare* $\cup$ {*inviter?*}
talkDesc' $\neq$ *talkDescDefault*
talkDescChanged' = *ztrue*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
wallDesc' = *wallDesc*
readDesc' = *readDesc*
wallDescChanged' = *wallDescChanged*
unreadCount' = *unreadCount*
talkPending' = *talkPending*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

17 Marker Consistency: the property and the defect

Write *talkActivity* for the disjunction of the three talk activity sources and *readActivity* for a positive unread count:

$$\begin{aligned}
 \text{talkActivity} &\hat{=}\text{ dom talkPending} \neq \emptyset \vee \text{talkBare} \neq \emptyset \vee \text{talkQueued} > 0 \vee \text{talkConnected} = \text{ztrue} \\
 \text{readActivity} &\hat{=}\text{ unreadCount} > 0.
 \end{aligned}$$

The intended invariant is that each marker is present exactly when its activity exists, and that a talk marker always carries a session-naming accept hint:

MarkerConsistent

NotificationState

(*talkDesc* $\neq$ *talkDescDefault*) $\Leftrightarrow$
($\text{dom talkPending} \neq \emptyset \vee \text{talkBare} \neq \emptyset \vee \text{talkQueued} > 0 \vee \text{talkConnected} = \text{ztrue}$)
(*readDesc* $\neq$ *readDescDefault*) $\Leftrightarrow$ *unreadCount* > 0

HintNamesSession

NotificationState

$talkDesc \neq talkDescDefault \Rightarrow talkBare = \emptyset$

Neither property is yet part of *NotificationState*, so both may fail. Two model-checking runs at set size 2 exhibit the failures as reachable states.

The stale marker. Setting the ProB goal to

$talkDesc \neq talkDescDefault \wedge \neg talkActivity$

finds a reachable state: an invite arrives (*TalkInviteArrive* or *TalkMessageArrive* raises the marker), then *TalkDrainStale* empties every activity source while preserving *talkDesc*. The marker persists with nothing behind it. This is the *biff-9la* defect.

The bare hint. Setting the goal to

$talkDesc \neq talkDescDefault \wedge talkBare \neq \emptyset$

finds a reachable state reached by *TalkInviteArriveBare*: the marker is present but the pending invite carries no session key, so its accept hint names no session.

Liveness. The safety goals above show a *bad state* is reachable. The sharper statement is a liveness one: from some reachable marker-present state, the marker can never clear. Expressed as the CTL formula

$AG (talkDesc \neq talkDescDefault \Rightarrow EF talkDesc = talkDescDefault),$

the model refutes it, because *TalkDrainStale* is the only way to empty activity and it does not touch *talkDesc*: once set, the marker is permanent. The liveness refutation is robust—it holds even though other operations remain enabled, so it is not masked the way a plain deadlock check would be.

18 The Fix: reconciling transitions

The repair has two parts. First, *MarkerConsistent*, *HintNamesSession* and the read biconditional are promoted into the state schema (Section 5), so they are now invariants that every operation must respect. Second, transitions are added that remove activity *and reconcile the description in the same step*.

Promoting the invariants has an immediate and welcome consequence: the two defect operations become infeasible. *TalkDrainStale* empties activity while preserving $talkDesc \neq talkDescDefault$; its post-state now contradicts *MarkerConsistent*, so it is enabled in no reachable state. *TalkInviteArriveBare* sets $talkBare \neq \emptyset$ while raising the marker; its post-state contradicts *HintNamesSession*, so it too is dead. ProB reports both as uncovered operations. This is the point of the exercise: once the consistency of marker and activity is an invariant, the specification *proves* that no drain can strand the marker and no invite can be recorded without its session key. The defect operations are kept in the text only as the negative results they now are.

A dead operation is not a live transition, however. Something must still remove activity, and—because the invariant forbids emptying activity without reverting the description—any such transition must do both at once. Four suffice: *TalkDrain*, *WithdrawArrive*, *ExpirePendingInvite* and *TalkAccept*; the proof in Section 19 shows each discharges an obligation the others do not. *WithdrawArrive* and *ExpirePendingInvite* together replace the single abstract *DropPendingInvite* of the earlier drafts: the two concrete removal paths the implementation must supply—a withdrawal frame and a time-to-live expiry—have distinct preconditions and so are kept distinct in the model.

The withdrawal path itself carries a further obligation the earlier drafts missed, which a second review found. A withdrawal frame names the invite it cancels only by the inviter’s user identity, but the pending invite is keyed by session: *talkPending* records the inviter’s *SESSIONKEY*. If *WithdrawArrive* removed on the bare user match, a withdrawal emitted by an earlier session could—because the talk transport gives no cross-session ordering—arrive after a fresh invite from a later session of the same user and silently delete the live invite. The guard $talkPending\ inviter? = withdrawKey?$ forecloses this: a withdrawal is

applied only when its originating session key matches the stored one. A frame whose key does not match is dropped by *WithdrawForeign*, the withdrawal-side analogue of *talk.tex*'s *DrainForeignAccept*. The username-only version is retained as the dead operation *WithdrawStale*, whose reachability under the missing guard, and unreachability once the guard is present, are the subject of the reordering proof in Section 19.

18.1 Draining messages and ending the conversation

TalkDrain empties the message queue and ends the connection. If no pending invite remains behind, it reverts the description; otherwise the marker stays, now backed only by the surviving invites.

<i>TalkDrain</i>
$\Delta \text{NotificationState}$
$ \begin{aligned} & \text{pendingAge}' = \text{pendingAge} \\ & (\text{talkQueued} > 0 \vee \text{talkConnected} = \text{ztrue}) \\ & \text{talkQueued}' = 0 \\ & \text{talkConnected}' = \text{zfalse} \\ & ((\text{dom } \text{talkPending} = \emptyset \wedge \text{talkBare} = \emptyset) \Rightarrow \\ & \quad (\text{talkDesc}' = \text{talkDescDefault} \wedge \text{talkDescChanged}' = \text{zfalse})) \\ & ((\text{dom } \text{talkPending} \neq \emptyset \vee \text{talkBare} \neq \emptyset) \Rightarrow \\ & \quad (\text{talkDesc}' = \text{talkDesc} \wedge \text{talkDescChanged}' = \text{talkDescChanged})) \\ & \text{natsConn}' = \text{natsConn} \\ & \text{kvProvisioned}' = \text{kvProvisioned} \\ & \text{talkSubscribed}' = \text{talkSubscribed} \\ & \text{session}' = \text{session} \\ & \text{toolContext}' = \text{toolContext} \\ & \text{wallDesc}' = \text{wallDesc} \\ & \text{readDesc}' = \text{readDesc} \\ & \text{wallDescChanged}' = \text{wallDescChanged} \\ & \text{unreadCount}' = \text{unreadCount} \\ & \text{talkPending}' = \text{talkPending} \\ & \text{talkBare}' = \text{talkBare} \\ & \text{queueItems}' = \text{queueItems} \\ & \text{queueKinds}' = \text{queueKinds} \\ & \text{queueTexts}' = \text{queueTexts} \\ & \text{currentIndex}' = \text{currentIndex} \\ & \text{fileExists}' = \text{fileExists} \\ & \text{fileUnreadCount}' = \text{fileUnreadCount} \\ & \text{fileItems}' = \text{fileItems} \\ & \text{barLine2}' = \text{barLine2} \\ & \text{barStale}' = \text{barStale} \\ & \text{lastNotifyResult}' = \text{lastNotifyResult} \\ & \text{pendingNotify}' = \text{pendingNotify} \\ & \text{notifyLost}' = \text{notifyLost} \\ & \text{inFlight}' = \text{inFlight} \\ & \text{beltPnSnapshot}' = \text{beltPnSnapshot} \\ & \text{capPnSnapshot}' = \text{capPnSnapshot} \\ & \text{raceLost}' = \text{raceLost} \\ & \text{staleWithdrawHit}' = \text{staleWithdrawHit} \end{aligned} $

18.2 Withdrawing a pending invite (ntWithdraw frame)

The earlier drafts collapsed the two ways a pending invite can be removed into a single abstract operation. The two mechanisms have genuinely different preconditions, however, and the implementation must provide both, so they are modelled here as distinct operations—the partition of Section 19 then owes a separate test for each.

WithdrawArrive is the first: the recipient receives an *ntWithdraw* notification from the inviter—the inviter ran *talk_end*, hung up, or otherwise cancelled—and removes that inviter's pending entry.

A withdrawal frame carries the inviter’s originating session key in the field *withdrawKey?*; this is the *ntWithdraw* analogue of the *nfromKey* that *talk.tex* guards on for accepts. The removal is honoured only when that key equals the session key stored against the pending invite—*inviter?* $\in \text{dom } \textit{talkPending}$ and *talkPending inviter?* = *withdrawKey?*. The guard is not decoration. The talk transport is core NATS publish and subscribe; it gives no cross-session ordering, so a withdrawal emitted by an earlier session of the same user may be delivered *after* a fresh invite from a later session of that user. A user invites us from session *A*, withdraws, and re-invites from session *B*; because *talkPending* is a function keyed on the user, the re-invite supersedes the first, so *talkPending* now maps that user to *B*’s key. A withdrawal keyed to session *A* that arrives late must not disturb the live session *B* invite—and with the key guard it does not, because the stored key (session *B*’s) differs from the withdrawal’s key (session *A*’s), so the equality *talkPending inviter?* = *withdrawKey?* fails and no removal occurs. Without the guard, the withdrawal would delete the current invite on a bare user match, the marker would clear, and the live invite would be silently lost. This is exactly the stale-marker defect class the specification exists to prevent, reached here by a non-adversarial reordering rather than by an attacker; the key guard is the withdrawal-side counterpart of the accept-side consent boundary of *talk.tex* (DES-043, invariant 11). A withdrawal whose key does not match is not applied here; it is handled by *WithdrawForeign* below. *WithdrawArrive*’s precondition is otherwise the arrival of the frame on a live connection; it does *not* depend on how long the invite has been pending. If no activity survives, the description reverts and the clock returns to zero.

WithdrawArrive

Δ *NotificationState*

inviter? : *USER*

withdrawKey? : *SESSIONKEY*

natsConn = *natsConnected*
inviter? $\in$ dom *talkPending*
talkPending inviter? = *withdrawKey?*
talkPending' = {*inviter?*} $\triangleleft$ *talkPending*
talkBare' = *talkBare*
 $((\text{dom } \text{talkPending}' = \emptyset \wedge \text{talkBare}' = \emptyset) \Rightarrow \text{pendingAge}' = 0)$
 $((\text{dom } \text{talkPending}' \neq \emptyset \vee \text{talkBare}' \neq \emptyset) \Rightarrow \text{pendingAge}' = \text{pendingAge})$
 $((\text{dom } \text{talkPending}' = \emptyset \wedge \text{talkBare}' = \emptyset \wedge$
 talkQueued = 0 $\wedge$ *talkConnected* = *zfalse*) $\Rightarrow$
 (*talkDesc'* = *talkDescDefault* $\wedge$ *talkDescChanged'* = *zfalse*))
 $((\text{dom } \text{talkPending}' \neq \emptyset \vee \text{talkBare}' \neq \emptyset \vee$
 talkQueued > 0 $\vee$ *talkConnected* = *ztrue*) $\Rightarrow$
 (*talkDesc'* = *talkDesc* $\wedge$ *talkDescChanged'* = *talkDescChanged*))
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
wallDesc' = *wallDesc*
readDesc' = *readDesc*
wallDescChanged' = *wallDescChanged*
unreadCount' = *unreadCount*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

18.3 A withdrawal that names the wrong session (*WithdrawForeign*)

WithdrawForeign is the withdrawal-side counterpart of *talk.tex*'s *DrainForeignAccept*. A withdrawal frame arrives whose key does *not* match the pending invite it purports to cancel—either the user has no pending invite at all, or the stored invite carries a different session key, as in the reordering above. The frame is dropped: the pending set, the marker, the clock, and every other component are left exactly as they were. Modelling it as a distinct operation, rather than as a silent no-op, forces the test-partition of Section 19 to owe a case for it, and records in the specification that the implementation must recognise and discard such a frame rather than acting on its user field alone.

WithdrawForeign

Δ *NotificationState*

inviter? : *USER*

withdrawKey? : *SESSIONKEY*

natsConn = *natsConnected*

$\neg (\text{inviter?} \in \text{dom talkPending} \wedge \text{talkPending inviter?} = \text{withdrawKey?})$

talkPending' = *talkPending*

talkBare' = *talkBare*

pendingAge' = *pendingAge*

talkDesc' = *talkDesc*

talkDescChanged' = *talkDescChanged*

natsConn' = *natsConn*

kvProvisioned' = *kvProvisioned*

talkSubscribed' = *talkSubscribed*

session' = *session*

toolContext' = *toolContext*

wallDesc' = *wallDesc*

readDesc' = *readDesc*

wallDescChanged' = *wallDescChanged*

unreadCount' = *unreadCount*

talkQueued' = *talkQueued*

talkConnected' = *talkConnected*

queueItems' = *queueItems*

queueKinds' = *queueKinds*

queueTexts' = *queueTexts*

currentIndex' = *currentIndex*

fileExists' = *fileExists*

fileUnreadCount' = *fileUnreadCount*

fileItems' = *fileItems*

barLine2' = *barLine2*

barStale' = *barStale*

lastNotifyResult' = *lastNotifyResult*

pendingNotify' = *pendingNotify*

notifyLost' = *notifyLost*

inFlight' = *inFlight*

beltPnSnapshot' = *beltPnSnapshot*

capPnSnapshot' = *capPnSnapshot*

raceLost' = *raceLost*

staleWithdrawHit' = *staleWithdrawHit*

18.4 The withdrawal gap, as a dead operation (*WithdrawStale*)

WithdrawStale is the defect this section exists to close, kept in the text as the negative result it now is. It models the username-only withdrawal: an *ntWithdraw* frame arrives whose key does not match the stored invite (*talkPending inviter?* $\neq$ *withdrawKey?*), yet it removes the invite anyway, on the bare user match. The removed invite is the live one; the marker clears with nothing behind it and the current invite is silently lost. The history component *staleWithdrawHit* records that this has happened, so the suppression can be named as a reachable state.

Promoting *staleWithdrawHit* = *zfalse* into *NotificationState* (Section 5) makes *WithdrawStale* infeasible: its post-state sets *staleWithdrawHit'* = *ztrue*, which the invariant forbids. ProB reports it as an uncovered operation, and the goal *staleWithdrawHit* = *ztrue* is unreachable. Delete the invariant—as an earlier, username-only draft implicitly did—and *WithdrawStale* becomes enabled: with two distinct session keys it drives the model to *staleWithdrawHit* = *ztrue*, the reordering counterexample of Section 19. This is the same device by which *TalkDrainStale* and *TalkInviteArriveBare* are proved dead against *MarkerConsistent* and *HintNamesSession*.

WithdrawStale

Δ *NotificationState*

inviter? : *USER*

withdrawKey? : *SESSIONKEY*

natsConn = *natsConnected*

inviter? $\in$ dom *talkPending*

talkPending *inviter?* $\neq$ *withdrawKey?*

talkPending' = {*inviter?*} $\triangleleft$ *talkPending*

talkBare' = *talkBare*

staleWithdrawHit' = *ztrue*

((dom *talkPending'* = $\emptyset$ $\wedge$ *talkBare'* = $\emptyset$) $\Rightarrow$ *pendingAge'* = 0)

((dom *talkPending'* $\neq$ $\emptyset$ $\vee$ *talkBare'* $\neq$ $\emptyset$) $\Rightarrow$ *pendingAge'* = *pendingAge*)

((dom *talkPending'* = $\emptyset$ $\wedge$ *talkBare'* = $\emptyset$ $\wedge$

talkQueued = 0 $\wedge$ *talkConnected* = *zfalse*) $\Rightarrow$

(*talkDesc'* = *talkDescDefault* $\wedge$ *talkDescChanged'* = *zfalse*))

((dom *talkPending'* $\neq$ $\emptyset$ $\vee$ *talkBare'* $\neq$ $\emptyset$ $\vee$

talkQueued > 0 $\vee$ *talkConnected* = *ztrue*) $\Rightarrow$

(*talkDesc'* = *talkDesc* $\wedge$ *talkDescChanged'* = *talkDescChanged*))

natsConn' = *natsConn*

kvProvisioned' = *kvProvisioned*

talkSubscribed' = *talkSubscribed*

session' = *session*

toolContext' = *toolContext*

wallDesc' = *wallDesc*

readDesc' = *readDesc*

wallDescChanged' = *wallDescChanged*

unreadCount' = *unreadCount*

talkQueued' = *talkQueued*

talkConnected' = *talkConnected*

queueItems' = *queueItems*

queueKinds' = *queueKinds*

queueTexts' = *queueTexts*

currentIndex' = *currentIndex*

fileExists' = *fileExists*

fileUnreadCount' = *fileUnreadCount*

fileItems' = *fileItems*

barLine2' = *barLine2*

barStale' = *barStale*

lastNotifyResult' = *lastNotifyResult*

pendingNotify' = *pendingNotify*

notifyLost' = *notifyLost*

18.5 Expiring a pending invite (time-to-live)

The second mechanism covers the inviter who never sends a withdrawal—a crash, a dropped connection, an abandoned session. No frame arrives, so *WithdrawArrive* is never enabled for that invite; the only thing that changes is the clock. *AgeTick* models the passage of time on a poller tick: while any invite is pending and the clock is below its bound, the clock advances by one. It touches nothing else—not the pending set, not the description, not the activity.

AgeTick

Δ *NotificationState*

```
toolContext = zfalse
(dom talkPending  $\neq \emptyset \vee$  talkBare  $\neq \emptyset$ )
pendingAge < maxInviteAge
pendingAge' = pendingAge + 1
natsConn' = natsConn
kvProvisioned' = kvProvisioned
talkSubscribed' = talkSubscribed
session' = session
toolContext' = toolContext
wallDesc' = wallDesc
talkDesc' = talkDesc
readDesc' = readDesc
wallDescChanged' = wallDescChanged
talkDescChanged' = talkDescChanged
unreadCount' = unreadCount
talkPending' = talkPending
talkBare' = talkBare
talkQueued' = talkQueued
talkConnected' = talkConnected
queueItems' = queueItems
queueKinds' = queueKinds
queueTexts' = queueTexts
currentIndex' = currentIndex
fileExists' = fileExists
fileUnreadCount' = fileUnreadCount
fileItems' = fileItems
barLine2' = barLine2
barStale' = barStale
lastNotifyResult' = lastNotifyResult
pendingNotify' = pendingNotify
notifyLost' = notifyLost
inFlight' = inFlight
beltPnSnapshot' = beltPnSnapshot
capPnSnapshot' = capPnSnapshot
raceLost' = raceLost
staleWithdrawHit' = staleWithdrawHit
```

ExpirePendingInvite then reaps an invite once its clock has reached the bound. Its precondition— $pendingAge = maxInviteAge$ on a poller tick—is distinct from *WithdrawArrive*’s, which fires on frame receipt at any age. The state effect is the same: the inviter’s entry is removed, and if no activity survives, the description reverts and the clock returns to zero.

ExpirePendingInvite

Δ *NotificationState*

inviter? : *USER*

toolContext = *zfalse*
pendingAge = *maxInviteAge*
(*inviter?* $\in$ dom *talkPending* $\vee$ *inviter?* $\in$ *talkBare*)
talkPending' = {*inviter?*} $\triangleleft$ *talkPending*
talkBare' = *talkBare* $\setminus$ {*inviter?*}
((dom *talkPending'* = $\emptyset$ $\wedge$ *talkBare'* = $\emptyset$) $\Rightarrow$ *pendingAge'* = 0)
((dom *talkPending'* $\neq$ $\emptyset$ $\vee$ *talkBare'* $\neq$ $\emptyset$) $\Rightarrow$ *pendingAge'* = *pendingAge*)
((dom *talkPending'* = $\emptyset$ $\wedge$ *talkBare'* = $\emptyset$ $\wedge$
 talkQueued = 0 $\wedge$ *talkConnected* = *zfalse*) $\Rightarrow$
 (*talkDesc'* = *talkDescDefault* $\wedge$ *talkDescChanged'* = *zfalse*))
((dom *talkPending'* $\neq$ $\emptyset$ $\vee$ *talkBare'* $\neq$ $\emptyset$ $\vee$
 talkQueued > 0 $\vee$ *talkConnected* = *ztrue*) $\Rightarrow$
 (*talkDesc'* = *talkDesc* $\wedge$ *talkDescChanged'* = *talkDescChanged*))
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
wallDesc' = *wallDesc*
readDesc' = *readDesc*
wallDescChanged' = *wallDescChanged*
unreadCount' = *unreadCount*
talkQueued' = *talkQueued*
talkConnected' = *talkConnected*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

18.6 Accepting an invite

TalkAccept is the ordinary path: a pending invite is consumed and the conversation becomes live. Activity does not vanish—it moves from *talkPending* to *talkConnected*—so the marker is preserved, consistent with the invariant.

TalkAccept

Δ *NotificationState*

inviter? : *USER*

inviter? $\in \text{dom } \text{talkPending}$
talkPending' = {*inviter?*} $\triangleleft \text{talkPending}$
talkConnected' = *ztrue*
 $((\text{dom } \text{talkPending}' = \emptyset \wedge \text{talkBare}' = \emptyset) \Rightarrow \text{pendingAge}' = 0)$
 $((\text{dom } \text{talkPending}' \neq \emptyset \vee \text{talkBare}' \neq \emptyset) \Rightarrow \text{pendingAge}' = \text{pendingAge})$
talkDesc' = *talkDesc*
talkDescChanged' = *talkDescChanged*
natsConn' = *natsConn*
kvProvisioned' = *kvProvisioned*
talkSubscribed' = *talkSubscribed*
session' = *session*
toolContext' = *toolContext*
wallDesc' = *wallDesc*
readDesc' = *readDesc*
wallDescChanged' = *wallDescChanged*
unreadCount' = *unreadCount*
talkBare' = *talkBare*
talkQueued' = *talkQueued*
queueItems' = *queueItems*
queueKinds' = *queueKinds*
queueTexts' = *queueTexts*
currentIndex' = *currentIndex*
fileExists' = *fileExists*
fileUnreadCount' = *fileUnreadCount*
fileItems' = *fileItems*
barLine2' = *barLine2*
barStale' = *barStale*
lastNotifyResult' = *lastNotifyResult*
pendingNotify' = *pendingNotify*
notifyLost' = *notifyLost*
inFlight' = *inFlight*
beltPnSnapshot' = *beltPnSnapshot*
capPnSnapshot' = *capPnSnapshot*
raceLost' = *raceLost*
staleWithdrawHit' = *staleWithdrawHit*

19 Marker Consistency: the proof

With the invariants promoted and the reconciling transitions in place, the checks that discharge the obligations are run at *DEFAULT_SETSIZE* 2 with *USER* shrunk to one and *SESSIONKEY* kept at two. Two session keys are essential here: the reordering defect turns on one user holding invites from two distinct sessions, and a single session key would collapse the two into one and hide the fault. One user suffices, because every marker property is a statement about the *aggregate* of the activity sources, not about how many users contribute to it; shrinking *USER* keeps the state space exhaustible while losing no distinction the properties depend on. At this scope the exploration reports *INITIALISATION*:8 against *SETUP_CONSTANTS*:8—the *MAX_INITIALISATIONS* bound never binds—and terminates with “all open states visited”: 598,824 states, 9,938,416 transitions, a complete check, not a truncated one.

```
probccli docs/notification.tex -model_check -noltl -coverage \  
-goal "staleWithdrawHit = ztrue" \  
-p DEFAULT_SETSIZE 2 \  
-card USER 1 -card SOURCEKEY 1 -card SESSIONKEY 2 \  
-card QSLLOT 1 -card USLOT 1 -card ASLOT 1 -card PSLOT 1 \  

```

-p MAX_INITIALISATIONS 16 -p MAX_OPERATIONS 8000

Safety. Ordinary model checking visits every reachable state and finds no invariant violation and no deadlock. Because *MarkerConsistent* and *HintNamesSession* are now part of the state schema, this establishes that no reachable state has a marker without its activity, and none has a talk marker whose hint names no session. The two goal predicates that were reachable in Section 17— $\text{talkDesc} \neq \text{talkDescDefault} \wedge \neg \text{talkActivity}$ and $\text{talkDesc} \neq \text{talkDescDefault} \wedge \text{talkBare} \neq \emptyset$ —are now unreachable, each being exactly the negation of a promoted invariant.

The reordering defect, reached and then closed. The withdrawal key guard is proved the same way the stale marker was. Delete the invariant $\text{staleWithdrawHit} = \text{zfalse}$ from the state schema, so that the username-only *WithdrawStale* becomes feasible, and set the ProB goal to $\text{staleWithdrawHit} = \text{ztrue}$. The check finds a reachable state by an eleven-step trace whose tail is the reordering exactly: *TalkInviteArrive*(u, k_B) records an invite from the user’s session B , then *WithdrawStale*(u, k_A)—a withdrawal carrying the earlier session A ’s key—removes the live invite on the bare user match, clearing the marker with the current invite silently lost. Restore the invariant, and the same goal is searched over the whole reachable space above: it is *not* found, and the check reports “all states visited.” *WithdrawStale* is then reported among the uncovered operations, alongside the earlier defect operations *TalkDrainStale* and *TalkInviteArriveBare*: the operation that would suppress a live invite exists in the text but is enabled in no reachable state. The key-matched *WithdrawArrive* and its foreign-drop counterpart *WithdrawForeign* are both covered.

The pending-set bound. This scope shrinks *USER* to one, which suffices for every marker property but cannot exercise *TalkInviteArriveOverflow*: that operation fires only when a *new* inviter arrives at a full pending set, and with a single user there is no second inviter to overflow it. It is therefore reported uncovered here, alongside the three dead defect operations—but for an entirely different reason. It is a live transition awaiting a second user, not a suppressed one. The exhaustive two-user check of the Global Constants section (*DEFAULT_SETSIZE* 2 with *PSLOT* at one, so $\text{maxPending} = 1$) covers it: one user fills the single pending slot, a second arrives and overflows it, *TalkInviteArriveOverflow* evicts the incumbent, and the bound $\# \text{talkPending} + \# \text{talkBare} \leq \text{maxPending}$ holds across all 598,825 reachable states with no invariant violation and no deadlock. That bound is the pending-set analogue of the queue cap maxQueueLen of *talk.tex*; it closes the unbounded-growth denial of service that a wire-controlled inviter key—each distinct forgery a permanent entry, the time-to-live being eventual rather than a cap—would otherwise permit.

Liveness. The CTL formula $\text{AG}(\text{talkDesc} \neq \text{talkDescDefault} \Rightarrow \text{EF } \text{talkDesc} = \text{talkDescDefault})$, checked at the same scope, is reported TRUE: from every marker-present state at least one reconciling transition is enabled and leads, directly or through *TalkAccept*, to a state in which the marker has cleared. The key guard does not weaken this—an invite keyed to k can still be cleared by a matching *WithdrawArrive*(u, k), and unconditionally by the time-to-live *ExpirePendingInvite*—so the channel remains self-clearing.

Necessity of each transition. Each reconciling transition discharges a distinct obligation. *TalkDrain* is the only transition that clears *talkQueued* or *talkConnected*; without it a marker backed by a queued message or a live connection can never clear. *WithdrawArrive* and *ExpirePendingInvite* are the two transitions that remove a pending invite without the recipient’s acceptance, and they are enabled under different conditions: *WithdrawArrive* on receipt of an *ntWithdraw* frame whose key matches the pending invite, from an inviter who cancels; *ExpirePendingInvite* on a poller tick once the clock has reached *maxInviteAge*. The second is what covers exactly the biff-9la scenario—an invite from a peer who never returns and never withdraws: only the time-to-live path clears it, so without *ExpirePendingInvite* (and the *AgeTick* that drives its clock) the talk channel would not be self-clearing in the way the read reference is. *WithdrawForeign* removes nothing; it is the transition that *declines* to act on a mismatched frame, and it is kept as an explicit operation so that the implementation owes a case for recognising and discarding such a frame rather than acting on its user field alone. *TalkAccept* is the ordinary progress transition; it is not required for the marker to clear, but without it the only exit from a pending invite is to remove it, and the handshake modelled in *talk.tex* could never complete.

20 System Properties

20.1 Notification Delivery Guarantee

Erratum. Earlier drafts stated *NotifyGuarantee* below as an unconditional invariant—“no operation produces *notifyDropped* after a description change.” *PollTickNotifyFail* and *PollTickNotifyFailUnrecorded* (Section 13) falsify it directly: both are enabled exactly when a description has changed, and both produce $\text{lastNotifyResult}' = \text{notifyDropped}$. The schema is kept below, unchanged, as a record of what was believed and is no longer claimed as a property of the whole reachable space. What *does* hold in the whole reachable space is the accounting safety property $\text{notifyLost} = \text{zfalse}$ (Section 13): a dropped send is not guaranteed never to happen, but it is guaranteed never to happen unrecorded.

<i>NotifyGuarantee</i>
<i>NotificationState</i>
$(\text{wallDescChanged} = \text{ztrue} \vee \text{talkDescChanged} = \text{ztrue}) \Rightarrow \text{lastNotifyResult} = \text{notifySent}$

What remains true, and a modelling seam worth naming: *WallPost* and *TalkStart* still set $\text{lastNotifyResult}' = \text{notifySent}$ unconditionally. This is not, on reflection, a claim that the belt path they use is somehow exempt from *NotifyBeltSendFail* (Section 10.1)—it is a scoping boundary of this specification. *WallPost*/*TalkStart* model the tool handler that mutates state and notifies *inline*, in one atomic step, as an abstraction of the whole request; *NotifyBelt*/*NotifyBeltBegin* model the underlying `notify_tool_list_changed()` call in isolation, as used by *KVWallReceive* and *NatsTalkCallback*’s downstream poller tick. Splitting *WallPost*/*TalkStart* the same way *NotifyBelt* was split is a straightforward, mechanical extension of this section’s technique should a future finding need it; it is out of scope here because no review has found a defect that requires it. *KVWallReceive* and *NatsTalkCallback* still defer the actual send to the poller, which is exactly the design that makes *PollTick*’s split (Section 13) the right place to model the suspenders-side failure.

20.2 Session Invariant—retracted

Erratum. Earlier drafts asserted *SessionInvariant* below as a state invariant and used it to prove *SuspendersAvailable*. Section 5’s erratum retracts it: *PollTickNotifyFail* sets $\text{session}' = \text{sessNone}$ while $\text{natsConn}' = \text{natsConnected}$, so the schema below is false of the corrected model and is not part of it. The property it was protecting—*EnsureConnected* cannot run before a first capture—is now *EnsureConnected*’s own explicit precondition (Section 8.1).

<i>SessionInvariant</i>
<i>NotificationState</i>
$\text{natsConn} = \text{natsConnected} \Rightarrow \text{session} = \text{sessCaptured}$

20.3 Suspenders Always Available—retracted

Erratum. For the same reason, *SuspendersAvailable* below no longer holds unconditionally: after *PollTickNotifyFail*, $\text{natsConn} = \text{natsConnected} \wedge \text{toolContext} = \text{zfalse}$ while $\text{session} = \text{sessNone}$, so the suspenders path is exactly *not* available until *CaptureSession*, *RecaptureSession*, *CaptureSessionFlushOk*, or a successful *NotifyBelt* call restores it. This is the substance of root cause #1: the property below was the promise the background poller made to itself, and it is the promise that broke.

<i>SuspendersAvailable</i>
<i>NotificationState</i>
$\text{natsConn} = \text{natsConnected}$
$\text{toolContext} = \text{zfalse}$
$\text{session} = \text{sessCaptured}$

What holds instead: whenever *SuspendersAvailable* fails with a changed flag set, $\text{notifyLost} = \text{zfalse}$ (the corrected model’s safety property) guarantees $\text{pendingNotify} = \text{ztrue}$, so the debt is visible to

whichever of *CaptureSession*, *RecaptureSession*, or *CaptureSessionFlushOk* arrives next—and, once Section 14’s mutex is in place, *raceLost* = *zfalse* guarantees that recovery is not itself clobbered by a second site’s concurrent, stale write.

20.4 Wall and Talk Symmetry

Wall posting uses the belt path (tool context) via *WallPost*, which sets *lastNotifyResult*' = *notifySent* unconditionally within this specification’s chosen scope (the modelling seam noted above the *NotifyGuarantee* erratum). Wall reception from another user uses *KVWallReceive*, and talk reception uses *NatsTalkCallback*; both defer the actual send to the poller rather than producing *notifySent* themselves, and that deferred send can now fail (Section 13). Section 13’s erratum to *SuspendersAvailable* applies to both deferred paths: the poller’s next tick attempts the send, and *notifyLost* = *zfalse* together with *raceLost* = *zfalse*, not delivery symmetry, is what guarantees the attempt is never silently and unrecoverably lost.

Erratum (biff-9la). The symmetry stated above concerns only *notification delivery*—that a `tools/list_changed` is sent. It does not extend to the *life-cycle of the tool description*, and the earlier drafts of this specification wrongly implied that it did by modelling *talkDesc* as an opaque value with no invariant. Wall and talk are not symmetric here. The wall and read markers are driven by sources that decrease—an unread count that falls to zero, a display item that is advanced away—so they clear themselves. The talk marker is driven by *talkPending*, *talkQueued* and *talkConnected*, and nothing in the original model reverted *talkDesc* when those emptied: Sections 17 reproduces the resulting stale marker as a reachable state. The corrected treatment requires the reconciling transitions of Section 18; only then does the talk description track its activity as the read reference does. The consistency invariant *MarkerConsistent*, not delivery symmetry, is the property that matters.

Erratum (*talk.tex* invariant 4). Invariant 4 of *talk.tex* (“Pending invites ... remain in the set until the next `talk` command”) describes exactly the grow-only behaviour that strands the marker. If pending invites are never removed except by a subsequent `talk` command, then a marker driven by *talkPending* can never clear on its own. Under the invariant *MarkerConsistent* that reading is superseded: a pending invite must also be removable by *WithdrawArrive*—an *ntWithdraw* frame from the inviter *whose session key matches the stored invite*—or by *ExpirePendingInvite*—a time-to-live expiry aged out on a poller tick—so that the talk channel is self-clearing. The key qualification on *WithdrawArrive* is not incidental: a withdrawal matched on the inviter’s user identity alone would, under the transport’s lack of cross-session ordering, let a stale withdrawal from one session delete a live invite from another, reintroducing the stranded-marker defect by a non-adversarial path. *WithdrawArrive* therefore guards on *talkPending inviter?* = *withdrawKey?*, and a mismatched frame is dropped by *WithdrawForeign*; this is the withdrawal-side mirror of the accept-side consent guard (*head queue*).*nfromKey* = *partnerKey* that *talk.tex*’s *DrainAcceptWhileInviting* already imposes (DES-043). The consent-boundary guarantees of *talk.tex* (invariant 11) are unaffected: withdrawal and expiry only *remove* pending invites, never complete a handshake, so they cannot forge consent.

WallNotifySuccess and *TalkNotifySuccess* describe the state immediately after a successful send—*PollTickNotifyOk*, or *NotifyBelt* before it clears the changed flag. Neither is a state invariant: a changed flag can also coexist with *lastNotifyResult* = *notifyDropped*, which is exactly *PollTickNotifyFail*’s and *PollTickNotifyFailUnrecorded*’s post-state (Section 13).

<i>WallNotifySuccess</i>	_____
<i>NotificationState</i>	
<i>wallDescChanged</i> = <i>ztrue</i>	
<i>lastNotifyResult</i> = <i>notifySent</i>	
<i>TalkNotifySuccess</i>	_____
<i>NotificationState</i>	
<i>talkDescChanged</i> = <i>ztrue</i>	
<i>lastNotifyResult</i> = <i>notifySent</i>	

20.5 Startup Sequence

The correct startup sequence is:

1. *Init* — all state zeroed, NATS disconnected, session uncaptured.
2. *CaptureSession* — session captured from server lifespan context.
3. *EnsureConnected* — NATS connected (*EnsureConnected*'s own precondition, Section 8.1, satisfied).
4. Poller starts, subscriptions begin — suspenders path available, until and unless a send failure strands it (Section 13).

EnsureConnected before any *CaptureSession* is rejected by its own guard $session \neq sessNone \vee natsConn = natsConnected$ (Section 8.1): the first disjunct fails because no capture has happened, and the second fails because nothing has connected yet. A *mid-session* loss of *session*, by contrast, does not block a repeat *EnsureConnected*—the second disjunct now holds, correctly, since the connection itself was never the problem.